



TRABAJO FIN DE GRADO
INGENIERÍA INFORMÁTICA

Control de navegación cazador-presa para sistemas aéreos no tripulados

Autor

Germán Rodríguez Vidal

Directores

Francisco Barranco Expósito
Álvaro Martínez Novo



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

Granada, Convocatoria de Junio curso 2025/26

Control de navegación cazador-presa para sistemas aéreos no tripulados

Autor

Germán Rodríguez Vidal

Directores

Francisco Barranco Expósito

Álvaro Martínez Novo



Departamento de

INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y ROBÓTICA

DEPARTAMENTO DE INGENIERÍA DE COMPUTADORES, AUTOMÁTICA Y
ROBÓTICA

—
Granada, Convocatoria de Junio, curso 2025/2026

Agradecimientos

Poner aquí agradecimientos.

Control de navegación cazador-presa para sistemas aéreos no tripulados

Germán Rodríguez Vidal

Palabras clave: Control de navegación, UAV, , Sistemas aéreos no tripulados, Navegación autónoma, Ardupilot, Gazebo, PID, Filtro de Kalman, Detección de Objetos

Resumen

Este Trabajo Fin de Grado propone el desarrollo de métodos de navegación autónoma para sistemas aéreos no tripulados (UAV, por sus siglas en inglés) convencionales en el caso de uso del problema cazador-presa. En concreto, se plantea una estrategia de control de navegación que permita el seguimiento a corta distancia de un UAV presa por parte de un UAV cazador, el cual debe perseguirlo en espacio abierto de forma estable y robusta. Para lograrlo, se integran métodos de control y técnicas de visión por computador que facilitan la detección, localización y seguimiento del objetivo durante la misión.

Los objetivos principales de este proyecto son, en primer lugar, validar un algoritmo de control de navegación capaz de realizar el seguimiento autónomo de un UAV presa a partir de la imagen. En segundo lugar, se persigue implementar los algoritmos desarrollados en una plataforma empotrada para su posterior validación experimental en condiciones de operación real.

Para alcanzar estos objetivos, se ha seleccionado el simulador 3D Gazebo como entorno de experimentación inicial. Esta herramienta permite evaluar los algoritmos de control en escenarios controlados, modificar variables de forma sistemática y repetir pruebas con alta precisión, lo que facilita una comparación rigurosa del rendimiento del sistema ante distintas trayectorias y condiciones dinámicas.

Una vez verificado el funcionamiento del algoritmo en simulación, mediante múltiples escenarios y perfiles de movimiento, se procederá a su validación en un campo de pruebas físico. En esta fase se analizará el comportamiento del sistema frente a situaciones reales, incluyendo perturbaciones e incertidumbres propias del entorno. El sistema se considerará satisfactorio si mantiene el seguimiento del UAV presa durante un tiempo predefinido y a una distancia fija respecto al objetivo.

Hunter-prey navigation control for unmanned aerial systems

Germán Rodríguez Vidal

Keywords: Navigation control, UAV, Unmanned aerial systems, Autonomous navigation, Ardupilot, Gazebo, PID, Kalman filter, Object detection

Abstract

This Bachelor's Thesis proposes the development of autonomous navigation methods for conventional unmanned aerial vehicles (UAVs) in the hunter-prey use case. Specifically, it presents a navigation control strategy that enables short-range tracking of a prey UAV by a hunter UAV, which must pursue it in open space in a stable and robust manner. To achieve this, control methods and computer vision techniques are integrated to facilitate target detection, localization, and tracking throughout the mission.

The main objectives of this project are, first, to validate a navigation control algorithm capable of autonomously tracking a prey UAV from image-based information and, second, to implement the developed algorithms on an embedded platform for subsequent experimental validation under real operating conditions.

To achieve these objectives, the 3D Gazebo simulator has been selected as the initial experimentation environment. This tool makes it possible to evaluate control algorithms in controlled scenarios, modify variables systematically, and repeat tests with high precision, which enables a rigorous comparison of system performance under different trajectories and dynamic conditions.

Once the algorithm has been verified in simulation through multiple scenarios and motion profiles, it will be validated in a physical test field. In this phase, the system behavior will be analyzed under real-world conditions, including disturbances and uncertainties inherent to the environment. The system will be considered satisfactory if it maintains tracking of the prey UAV for a predefined period and at a fixed distance from the target.

Yo, **Germán Rodríguez Vidal**, alumno de la titulación Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 32913011B, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Germán Rodríguez Vidal

Granada a Junio de 2026 .

D. **Francisco Barranco Expósito (tutor1)**, Profesor del Área de Arquitectura y Tecnología de Computadores del Departamento Ingeniería de Computadores, Automática y Robótica de la Universidad de Granada.

D. **Álvaro Martínez Novo (tutor2)**, Profesor del Área de XXXX del Departamento YYYY de la Universidad de Granada.

Informan:

Que el presente trabajo, titulado ***Control de navegación cazador-presa para sistemas aéreos no tripulados***, ha sido realizado bajo su supervisión por **Germán Rodríguez Vidal** , y autorizamos la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expiden y firman el presente informe en Granada a Junio de mes de 2026 .

Los directores:

Francisco Barranco Expósito

Álvaro Martínez Novo

Índice general

Índice de figuras	5
Índice de tablas	6
Índice de extractos de código	7
1. Introducción	8
1.1. Descripción del problema	8
1.2. Motivación	10
1.3. Objetivos	11
1.3.1. Objetivo General	11
1.3.2. Objetivos Específicos	11
1.4. Planificación	12
1.4.1. Detalle de las fases	12
1.4.2. Presupuesto	14
2. Estado del Arte	16
2.1. Introducción	16
2.2. Sistemas Aéreos no Tripulados	17
2.2.1. Clasificación por configuración física	17
2.2.2. Aplicaciones actuales	19
2.3. Métodos de Control de Navegación para UAVs	22
2.3.1. Tipos de control de navegación	23
2.3.2. Conceptos de Visual Servoing (IBVS)	24
2.4. Deep Learning	25
2.5. Modelos de regresión	26
2.6. Visión por Computador	27
2.6.1. Paradigmas de detección de objetos	28
2.6.2. YOLO	29
2.7. Geometría Proyectiva	30
2.7.1. Cámara estenopeica	30
2.7.2. Modelo geométrico de la cámara	32
2.7.3. Usos en navegación visual	32
2.8. Métodos de Control	33
2.8.1. Sistemas de Lazo Abierto vs. Lazo Cerrado	33
2.8.2. Controlador PID	34
2.9. Estimación de Estado y Filtrado	35
2.9.1. Origen y Evolución del Filtro de Kalman	35
2.9.2. Fundamentos y Mecanismo Recursivo	36
2.9.3. Aplicación en Persecución Visual y Alta Velocidad	36

2.9.4.	Filtro EMA	37
2.10.	Stack de Software y Hardware	38
2.10.1.	Comunicación Distribuida en Robótica	39
2.10.2.	Computación Distribuida Seleccionada	40
2.10.3.	Simulación	45
2.10.4.	Simulador Seleccionado	46
3.	Metodología de Trabajo	48
3.1.	Configuración de ROS	48
3.1.1.	Creación de un Espacio de Trabajo	48
3.1.2.	Instalación de Paquetes	49
3.2.	Configuración y Ejecución de la Simulación en Gazebo	51
3.2.1.	Modelo del Vehículo Aéreo y Configuración del Entorno	51
3.2.2.	Lanzamiento y Supervisión de la Simulación	52
3.3.	Configuración UAVs en Simulación	53
3.3.1.	Configuración del Dron 1	53
3.3.2.	Configuración del Dron 2	56
3.3.3.	Ejecución Básica de la Simulación	58
3.4.	Configuración del Control Propuesto y Ejecución en Gazebo	60
4.	Implementacion	61
4.1.	Solucion Software	61
4.1.1.	Preparación del Espacio de Trabajo	61
4.1.2.	Solución para Simulación	61
4.1.3.	Desarrollo de Scripts y Gráficas	61
4.2.	Arquitectura del Sistema	61
4.3.	Modulo de percepcion	61
4.3.1.	Eleccion de modelo de Yolo	61
4.3.2.	USo Yolo	61
4.3.3.	Uso de Filtro de Kalman y EMA	61
4.4.	Geometría Proyectiva	61
4.4.1.	Fundamentos de la Geometría Proyectiva	62
4.4.2.	Parámetros intrínsecos, FOV y distorsión	63
4.5.	Filtro de Kalman	65
4.6.	Compensacion de Actitud y proyeccion	68
4.7.	Control de Seguimiento	68
4.7.1.	Control PID	68
4.7.2.	Control de distancia	68
4.8.	Configuración del Control Propuesto y Ejecución en Gazebo	68
4.9.	Metricas	68

5. Pruebas y Validaciones	69
5.1. Planificación de las Pruebas	69
5.2. Métricas de Evaluación	69
5.3. Desarrollo de los Experimentos	69
5.3.1. Seguimiento del Objetivo	69
5.4. Análisis de Resultados	69
5.5. Experimentos	69
6. Conclusiones	70
6.1. Revisión del cumplimiento de objetivos	70
6.2. Conclusiones finales	70
6.3. Futuros Trabajos	70
7. Bibliografía	71

Índice de figuras

1.1. Logo del INTA	9
2.1. Dron de ala fija	18
2.2. Dron multirrotor	18
2.3. Dron VTOL de ala fija	19
2.4. Dron monitoreando cultivos	20
2.5. Dron utilizado en Busqueda y rescate Marítimo	20
2.6. Dron utilizado transporte de paquetes	21
2.7. Drones utilizados para operaciones militares	21
2.8. Tipos de algoritmos de visual Servoing (Elaboración propia)	23
2.9. Esquema general de una red neuronal multicapa	26
2.10. Ejemplo ilustrativo de detección de objetos mediante cajas delimitadoras	28
2.11. Esquema simplificado del funcionamiento de R-CNN	29
2.12. Esquema simplificado del funcionamiento de Yolo	30
2.13. Funcionamiento simplificado de una cámara estenopeica	31
2.14. Esquema simplificado del modelo de cámara estenopeica	32
2.15. Esquema de un PID	34
2.16. Efecto del filtro EMA sobre una señal ruidosa	38
2.17. Conexión de Nodos durante ejecución(Imagen de Elaboración Propia)	40
2.18. Logo de ROS	42
2.19. Uso de Gazebo con la camara. (Imagen de Elaboración Propia)	47
3.1. Panel básico de ejecución con varias terminales organizadas en Terminator	58
3.2. Marcos de referencia publicados durante la ejecución inicial con dos UAVs	59
4.1. Esquema simplificado del modelo de cámara estenopeica	62
4.2. sDistorsion Camera pinhole	64
4.3. Ejemplo uso del Filtro de Kalman en dron real , en modo predicción (Elaboración propia)	65

Índice de tablas

1.1. Planificación del proyecto	12
1.2. Resumen de costes del proyecto	15

Índice de extractos de código

3.1.	Archivo <code>apm.launch</code> para la conexión de <i>MAVROS</i>	52
3.2.	Extracto de <code>model.config</code> para declarar el modelo <code>drone1</code> .	53
3.3.	Extracto de código en <code>model.sdf</code> , sensor de cámara incorporado para <code>drone1</code>	54
3.4.	Identificador asignado al primer UAV en <code>drone1_params.parm</code>	55
3.5.	Fragmento principal de <code>apm_config_d1.yaml</code> para <code>drone1</code> . .	55
3.6.	Extracto de <code>model.config</code> para declarar el modelo <code>drone2</code> .	56
3.7.	Identificador asignado al segundo UAV en <code>drone2_params.parm</code>	56
3.8.	Fragmento principal de <code>apm_config_d2.yaml</code> para <code>drone2</code> . .	56
3.9.	Inclusión de <code>drone1</code> y <code>drone2</code> en <code>runaway.world</code>	57

Introducción

1.1. Descripción del problema

El auge y la proliferación de los Vehículos Aéreos No Tripulados (UAVs), especialmente los cuadricópteros, ha transformado múltiples sectores que van desde la logística hasta la defensa, gracias a sus capacidades de despegue y aterrizaje vertical (VTOL), alta maniobrabilidad y coste relativamente bajo. Sin embargo, esta democratización tecnológica ha generado simultáneamente nuevos retos de seguridad debido a su potencial uso hostil, irrumpiendo cada vez más en espacios aéreos restringidos como aeropuertos, zonas militares e infraestructuras críticas

Las contramedidas actuales para neutralizar estas amenazas, que incluyen sistemas de interferencia electrónica y soluciones cinéticas (como láseres o torretas), presentan limitaciones significativas debido a los riesgos de daños colaterales, restricciones regulatorias y altos costes operativos. En este contexto, los cuadricópteros han surgido como una plataforma óptima para la interceptación. A diferencia de los sistemas estacionarios o de superficie, un dron interceptor opera en el mismo dominio aéreo que su objetivo, ofreciendo un mayor alcance y precisión, a la vez que minimiza las interrupciones involuntarias al tráfico aéreo legal. Además, su despliegue resulta mucho más rentable en comparación con los sistemas tradicionales de grado militar, presentándose como una solución escalable para asegurar espacios aéreos sensibles

No obstante, la eficacia de un dron cazador depende críticamente de su nivel de autonomía. El enfoque convencional de operación remota basa su éxito en la pericia del piloto humano bajo estrés y en la integridad de un enlace de radiocomunicación bidireccional en tiempo real. Esta dependencia hace que los sistemas sean detectables, limitando su escalabilidad y tiempos de respuesta. Peor aún, los vuelve extremadamente vulnerables a la latencia, a técnicas de guerra electrónica y a la inhibición de frecuencias (jamming o spoofing de GPS), amenazas predominantes que pueden dejar al cazador inoperativo en el momento crítico de la misión.

Para superar estas vulnerabilidades, este Trabajo Fin de Grado aborda el desarrollo de una solución de navegación autónoma para un escenario de seguimiento a corta distancia de tipo cazador-presa. Se propone una estrategia de control basada en la identificación y persecución visual activa, permitiendo al UAV cazador aproximarse a su objetivo de forma precisa. Frente a la vulnerabilidad de las comunicaciones, esta solución garantiza la independencia táctica al procesar la información visual mediante algoritmos de inteligencia artificial directamente en una plataforma empotrada (on-board) en el dron. Esto permite tiempos de reacción mínimos y un seguimiento robusto que no se ve degradado por interferencias, maximizando la eficiencia de la respuesta.

A su vez, para que esta solución mantenga su viabilidad económica frente a alternativas que utilizan sensores costosos (como LiDAR o radar) que erosionan la rentabilidad de los UAVs pequeños, el sistema propuesto está diseñado para operar con un conjunto sensorial mínimo: una cámara monocular y una unidad de medida inercial (IMU).

Esta investigación se enmarca directamente en la necesidad de fortalecer la soberanía tecnológica nacional en materia de seguridad y defensa. En esta línea, el desarrollo está alineado con los intereses del **Instituto Nacional de Técnica Aeroespacial (INTA)**, con sede principal en Torrejón de Ardoz (Madrid), orientado al desarrollo de capacidades avanzadas para la detección, monitorización y neutralización de UAVs que operen de forma hostil. Uno de los objetivos de ese proyecto es precisamente el abordado en este TFG.



Figura 1.1. Logo del INTA.

Por ello, el problema específico abordado en este trabajo es demostrar que un sistema de control de navegación visual, basado en modelos de percepción profunda (Deep Learning) y algoritmos de control reactivo, permite a un UAV cazador realizar una persecución autónoma, estable y robusta en tiempo real. Los criterios principales a validar durante la investigación son:

- Mantener al UAV objetivo enfocado y centrado en el campo de visión de la cámara de forma continua, minimizando el error de seguimiento.

- Asegurar la capacidad de respuesta y maniobrabilidad del UAV cazador ante trayectorias dinámicas, aceleraciones bruscas o movimientos imprevistos de la presa.
- Garantizar que el stack de software pueda ejecutarse manteniendo una tasa de fotogramas (FPS) óptima en hardware de recursos limitados.

Para llevar a cabo esta validación, el sistema propuesto se probará inicialmente en entornos de simulación 3D, utilizando ROS y Gazebo, integrados con ArduPilot mediante simulaciones SITL. Finalmente, tras analizar las métricas obtenidas, se preparará su despliegue en plataformas empotradas reales para su evaluación práctica.

1.2. Motivación

El uso en aumento de sistemas aéreos no tripulados en defensa con la amenaza que representan los UAVs para la seguridad nacional hace necesaria la creación de sistemas antidrón que funcionen de forma autónoma, eficaz y segura. A medida que los UAVs comerciales son más accesibles y potentes, también se incrementa el riesgo de un uso indebido: desde espiar hasta amenazar a infraestructuras críticas, áreas urbanas y eventos donde hay una gran concentración de personas. Esto ha evidenciado la apremiante necesidad de tener sistemas de neutralización reactivos y muy eficaces.

La principal motivación de este trabajo es dotar a los UAVs cazadores de la capacidad de percepción y navegación que permitan adaptarse rápidamente a maniobras evasivas o a cambios inesperados en las trayectorias de los objetivos. Se trata de ir más allá de las limitaciones de los clásicos métodos de navegación basados en control remoto o en la recepción continua de coordenadas GPS, y de ofrecer una solución basada en modelos de Deep Learning y en algoritmos de control reactivo basados en visión artificial.

Asimismo, el desafío tecnológico de combinar un sistema de detección visual con un control de navegación avanzado en un entorno de simulación, junto con su implementación en hardware real, ofrece una práctica valiosa para aplicar los conocimientos adquiridos. Esto implica integrar disciplinas como la inteligencia artificial, la teoría del control, la simulación robótica y el desarrollo de sistemas embebidos. Estas áreas son clave en la ingeniería informática y en la robótica moderna.

Este trabajo no solo busca mejorar la autonomía y precisión de seguimiento de los UAVs cazadores, sino también establecer las bases para desarrollos futuros en navegación autónoma robusta, que opere de manera independiente en el exigente campo de la seguridad y defensa.

1.3. Objetivos

1.3.1. Objetivo General

Desarrollar e implementar un sistema robusto de control de navegación autónoma para vehículos aéreos no tripulados (UAVs) capaz de realizar el seguimiento de un objetivo en tiempo real, integrando técnicas de control reactivo, algoritmos de estimación de estado y visión por computador.

1.3.2. Objetivos Específicos

Para alcanzar el objetivo general, se proponen los siguientes objetivos específicos:

- **OE1: Evaluar y optimizar modelos de percepción visual para sistemas empuotrados.**Entrenar y comparar distintas arquitecturas de detección de objetos de última generación (familia YOLO) evaluando el equilibrio (trade-off) entre precisión matemática (mAP) , tiempo de inferencia y tamaño del modelo para seleccionar el modelo óptimo que garantice un seguimiento fluido en tiempo real.
- **OE2: Diseñar e implementar el lazo de control de navegación visual.**Desarrollar el algoritmo encargado de traducir las detecciones 2D de la cámara en comandos de vuelo 3D. Esto implicará el uso de transformaciones geométricas de coordenadas y la sintonización de un controlador reactivo (PID) para mantener a la presa centrada en el campo de visión.
- **OE3: Integrar la arquitectura software en un entorno de simulación realista.**Implementar el sistema completo utilizando ROS como middleware de comunicación, acoplando los nodos de visión y control con el simulador físico 3D Gazebo y ArduPilot mediante simulaciones SITL (Software In The Loop).
- **OE4: Validar la robustez del sistema frente a maniobras evasivas.**Diseñar y ejecutar una batería de pruebas automatizadas donde el UAV objetivo (presa) realice diferentes perfiles de vuelo dinámicos (slalom, espirales, frenadas de emergencia) para cuantificar el error de seguimiento y los tiempos de respuesta del cazador.
- **OE5: Desplegar y evaluar la viabilidad computacional en una plataforma empuotrada real.** Migrar el stack de software desarrollado a un hardware de recursos limitados para verificar que el sistema es capaz de mantener la tasa de procesamiento necesaria (FPS) sin comprometer la latencia ni la seguridad del vuelo.

1.4. Planificación

La planificación del proyecto se llevó a cabo utilizando un método en cascada, siguiendo un proceso lineal donde cada etapa se basa en el éxito de la precedente. Este enfoque es el adecuado, ya que cada progreso tecnológico debe consolidar la fase anterior para reducir riesgos y fallos.

El tiempo total para la realización del proyecto se estimó en 300 horas, correspondientes a 12 créditos ECTS. Sin embargo, la asignación de este tiempo experimentó algunas alteraciones debido a la necesidad de ajustarse a otras responsabilidades personales y a los obstáculos técnicos que surgieron a lo largo del desarrollo.

En la tabla que se presenta a continuación se puede apreciar claramente la organización general del proyecto, seguida de una explicación detallada de cada etapa.

Fase	Duración	Oct.	Nov.	Dic.	Ene.	Feb.	Mar.	Abr.	May.
Configuración del Entorno e Integración del Controlador	1 mes	X							
Incorporación de los UAVs al Simulador	1 mes		X						
Configuración de los Controladores	2 meses			X	X				
Pruebas Simuladas y Representación Visual de los Resultados	2 meses					X	X		
Implementación del Control en UAV real	1 mes							X	
Documentación	3 semanas								X

Tabla 1.1: Planificación del proyecto.

1.4.1. Detalle de las fases

A continuación, se describe brevemente el contenido de cada una de las etapas representadas en la Tabla 1.1.

- 1. Configuración del entorno e integración del controlador (octubre).** Durante esta fase se instaló y configuró el entorno de trabajo, incluyendo ROS, Gazebo, MAVROS y ArduPilot.
- 2. Incorporación de los UAVs al simulador (noviembre).** Se integraron los modelos de los UAVs en Gazebo y se verificó el funcionamiento de los sensores virtuales y de los canales de comunicación. Esta etapa permitió disponer de un entorno de simulación consistente y representativo del escenario de trabajo.
- 3. Configuración de los controladores (diciembre-enero).** Se desarrollaron y ajustaron los controladores necesarios para el seguimiento

visual del objetivo, así como los nodos ROS encargados de intercambiar información entre percepción, estimación y actuación. Además, se realizaron pruebas iniciales en condiciones controladas para comprobar la estabilidad del lazo de control.

4. **Pruebas simuladas y representación visual de los resultados (febrero–marzo).** Se ejecutaron vuelos de prueba en distintos escenarios simulados para evaluar métricas como el error de seguimiento, la capacidad de respuesta y la estabilidad del sistema frente a maniobras dinámicas de la presa. Los datos obtenidos se procesaron y representaron mediante gráficas que facilitaron su análisis e interpretación.
5. **Implementación del control en UAV real (abril).** El sistema desarrollado se adaptó para su despliegue en una plataforma embebida asociada al UAV real. En esta fase se realizaron ajustes de integración y las primeras verificaciones experimentales orientadas a validar el funcionamiento del sistema fuera del entorno puramente simulado.
6. **Documentación (mayo, 3 semanas).** Finalmente, se elaboró la memoria del proyecto, recopilando el desarrollo realizado, los resultados obtenidos, las conclusiones principales y las posibles líneas de mejora y trabajo futuro.

Si bien la planificación inicial resultó adecuada, durante el desarrollo del proyecto surgieron desviaciones inevitables respecto al cronograma previsto. En particular, la fase de configuración del entorno requirió más esfuerzo del estimado debido a incompatibilidades entre distintas versiones de ROS y a dificultades en la integración con MAVROS. Asimismo, la comunicación entre los UAVs presentó problemas adicionales que obligaron a realizar ajustes sucesivos en la arquitectura software y provocaron retrasos puntuales en algunas de las fases posteriores.

Las pruebas en simulación se desarrollaron de manera satisfactoria. No obstante, la incorporación de nuevas implementaciones y ajustes de diseño prolongó ligeramente esta fase, ya que durante su ejecución se detectaron pequeños errores que fue necesario corregir de forma iterativa. Aun así, el desarrollo de scripts y automatizaciones para el procesamiento y la representación de los datos contribuyó a agilizar el análisis de resultados y a hacer más eficiente el flujo de trabajo.

En conjunto, la metodología en cascada permitió mantener una estructura de trabajo clara y ordenada a lo largo del proyecto. No obstante, su carácter secuencial dificultó en determinados momentos la adaptación ágil ante incidencias imprevistas, especialmente durante las fases de integración

de módulos y depuración del entorno experimental. A pesar de estas limitaciones, el proyecto avanzó conforme a los plazos generales previstos, y los resultados obtenidos en simulación respaldan la viabilidad del enfoque de navegación autónoma propuesto como base para su posterior validación en una plataforma real.

1.4.2. Presupuesto

El desarrollo del proyecto ha implicado distintos costes asociados al uso de equipamiento informático para la ejecución de simulaciones, a la adquisición de materiales necesarios para la implementación del sistema embebido en el UAV real y al coste estimado de la dedicación personal. A continuación, se detallan los conceptos principales que conforman el presupuesto global del proyecto.

Equipo y herramientas

Durante las fases iniciales del trabajo fue necesario disponer de un equipo informático con capacidad suficiente para ejecutar simulaciones con una carga moderada tanto de procesamiento como gráfica. Para ello se utilizó un ordenador personal, cuyas prestaciones procesador de alto rendimiento, tarjeta gráfica dedicada y memoria RAM suficiente permitieron ejecutar de manera adecuada los entornos ROS, Gazebo y el resto de herramientas software empleadas en el desarrollo.

El coste estimado del equipo informático utilizado se fija, de forma aproximada, en **1.400€**.

Gastos de implementación del sistema embebido

La implementación del sistema embebido en el UAV real supuso un coste total de **2.693,10€**. Este importe incluye la adquisición de componentes estructurales, como el chasis y el tren de aterrizaje; elementos de propulsión, entre ellos motores, variadores electrónicos de velocidad (ESC) y hélices; el sistema de control y navegación, compuesto por la controladora Pixhawk Cube Orange, el módulo RTK y la estación base; los dispositivos de comunicación, como la telemetría RFD y la emisora RC; el sistema de procesamiento embebido basado en una Jetson Orin Nano; los elementos de alimentación, como baterías y convertidores DC-DC; así como cableado, conectores y diverso material auxiliar de montaje.

Horas de trabajo

La dedicación al proyecto se estima en 300 horas, correspondientes a los 12 créditos ECTS asignados. Para realizar una valoración económica orientativa del trabajo desarrollado, se considera una tarifa de 20€ por hora, representativa de un perfil junior de ingeniería [3]. Bajo esta hipótesis, el coste asociado a la dedicación personal asciende a:

$$300 \text{ horas} \times 20\text{€} / \text{hora} = 6000\text{€}$$

Resumen de costes

En la Tabla 1.2 se recoge el resumen de los costes estimados del proyecto.

Concepto	Coste (€)
Ordenador personal para simulaciones	1.400,00
Materiales para la implementación del sistema embebido	2.693,10
Horas de trabajo	6.000,00
Total estimado del proyecto	10.093,10

Tabla 1.2: Resumen de costes del proyecto.

Por tanto, el coste total estimado del proyecto asciende a **10.093,10€**, considerando tanto los recursos materiales empleados como la valoración económica aproximada del tiempo de trabajo dedicado a su desarrollo.

Estado del Arte

2.1. Introducción

Esta sección tiene como objetivo establecer el marco teórico y tecnológico que sustenta el desarrollo de este proyecto. Dado que el control de navegación para el seguimiento a cortas distancias de vehículos aéreos no tripulados (UAV) constituye un campo multidisciplinar, resulta necesario revisar las principales tecnologías empleadas para abordar este problema [29] y fundamentar la solución propuesta.

En primer lugar, se revisan los Sistemas Aéreos no Tripulados, analizando su clasificación actual y cómo su evolución ha permitido que su utilización se extienda globalmente, pasando de aplicaciones militares a complejas tareas de investigación y servicios civiles [7, 8].

El bloque de percepción visual introduce, en primer lugar, los fundamentos del *Deep Learning* y de los modelos de regresión, para después enmarcarlos dentro de la visión por computador y de la detección de objetos. En este contexto, se analiza la familia YOLO como uno de los enfoques más extendidos para percepción visual en tiempo real sobre plataformas embarcadas [50].

El núcleo estratégico del proyecto se desarrolla en el Control de Navegación Visual. Se analizan los distintos tipos de navegación aérea, profundizando en el paradigma IBVS (Image-Based Visual Servoing) [27]. Basándose en la literatura técnica consultada, se justifica el uso de IBVS como una solución más eficiente y menos costosa en términos de procesamiento y complejidad técnica que otros enfoques [29, 31]. Asimismo, se exploran diversos métodos de control, fundamentando la elección del controlador PID como una herramienta clásica y eficaz para el seguimiento de objetivos dinámicos.

Dada la naturaleza ruidosa de las detecciones visuales, la sección de Estimación de Estado y Filtrado detalla el uso del Filtro de Kalman [52] y el filtro EMA (Exponential Moving Average). Estas herramientas son críticas para mitigar el ruido de los sensores y predecir la posición del objetivo en caso de oclusiones temporales.

Para conectar el plano de la imagen con el mundo físico, se presentan los fundamentos de la Geometría Proyectiva [63]. En este apartado se explica el

funcionamiento de la proyección de la cámara y el uso de transformaciones de coordenadas, esenciales para traducir los errores en píxeles a comandos de movimiento en el sistema de referencia del dron [58].

Finalmente, se describe el Stack de Software y Hardware, detallando los frameworks de comunicación robótica (ROS/Mavros) y las herramientas de simulación (Gazebo) que permiten la validación segura de los algoritmos desarrollados .

2.2. Sistemas Aéreos no Tripulados

Los Sistemas Aéreos no Tripulados (UAS, por sus siglas en inglés, *Unmanned Aerial Systems*) han experimentado una evolución notable en las últimas décadas. Lo que comenzó como una tecnología asociada principalmente al ámbito militar se ha transformado en una plataforma versátil con aplicaciones en ingeniería civil, agricultura de precisión, logística, inspección y vigilancia. Esta expansión ha sido posible gracias a la miniaturización de sensores y al incremento de la capacidad de cómputo embarcado, lo que ha permitido desplegar drones en escenarios complejos donde la autonomía constituye un requisito crítico .

Sin embargo, el diseño de sistemas de navegación autónoma que sean robustos, eficientes y capaces de adaptarse a entornos dinámicos y no estructurados sigue siendo uno de los principales retos actuales.

2.2.1. Clasificación por configuración física

Atendiendo a los requisitos de maniobrabilidad y a la naturaleza de la misión, los UAS pueden clasificarse principalmente según su estructura [7]:

- **Ala fija (*Fixed-wing*).** Presentan una arquitectura similar a la de la aviación convencional. Destacan por su elevada autonomía y eficiencia en vuelos de larga distancia. No obstante, su incapacidad para realizar vuelo estacionario (*hover*) los hace poco adecuados para misiones de interceptación que requieran cambios bruscos de dirección o seguimiento cercano a baja velocidad. La Figura 2.1 muestra un ejemplo representativo de esta configuración.



Figura 2.1. Dron de ala fija. Fuente: [4].

- **Multirrotores (*Multi-rotors*).** Constituyen la plataforma de referencia para este Trabajo de Fin de Grado. Su configuración, habitualmente basada en cuatro motores, permite un control directo. Su capacidad de despegue y aterrizaje vertical (VTOL), junto con su elevada agilidad, resulta esencial para mantener a un objetivo móvil dentro del campo de visión de la cámara de forma continua. La Figura 2.2 presenta un ejemplo de esta configuración.



Figura 2.2. Dron multirrotor. Fuente: [6].

- **Híbridos (VTOL *Fixed-wing*).** Tratan de combinar la autonomía del ala fija con la versatilidad del multirrotor, aunque su mayor complejidad mecánica y de control suele traducirse en un incremento del peso, el coste y la dificultad de integración. La Figura 2.3 presenta un ejemplo de plataforma VTOL de ala fija.



Figura 2.3. Dron VTOL de ala fija. Fuente: [5].

2.2.2. Aplicaciones actuales

Entre sus distintas configuraciones, el dron multirrotor se ha consolidado como la plataforma más extendida debido a su maniobrabilidad, su facilidad de despliegue y su capacidad de vuelo estacionario, cualidades especialmente valiosas en operaciones que requieren cercanía al entorno y una respuesta rápida ante cambios del escenario.

Tradicionalmente, muchas de estas operaciones han dependido de un control humano directo, en el que un operador supervisa la trayectoria, corrige desviaciones y decide cada maniobra relevante. Este enfoque puede resultar suficiente en escenarios simples o muy controlados; sin embargo, pierde eficacia cuando el entorno es dinámico, cuando aparecen obstáculos imprevistos o cuando el objetivo modifica su trayectoria de forma continua.

En este contexto, la navegación autónoma surge como una solución tecnológica necesaria no solo para mejorar el rendimiento del sistema, sino también para reducir la carga de trabajo del operador humano. En misiones prolongadas, repetitivas o de alta exigencia, la dependencia constante del pilotaje manual incrementa el riesgo de fatiga, errores de supervisión y pérdida de eficacia operativa. La incorporación de capacidades autónomas permite automatizar tareas como el seguimiento de trayectorias, la corrección del movimiento y la respuesta ante cambios del entorno, disminuyendo la intervención continua del operador. Además, este enfoque contribuye a optimizar recursos y a abaratar costes operativos, al reducir la necesidad de control manual permanente y favorecer misiones más estables, seguras y sostenibles en el tiempo.

A partir de esta idea, la navegación autónoma adquiere especial relevancia en varias situaciones de uso:

Utilización en agricultura: En el ámbito de la agricultura de precisión, la navegación autónoma permite recorrer parcelas extensas, mantener trayectorias estables y adaptar el vuelo a las características del terreno o del cultivo [19]. Esto facilita tareas como la supervisión del estado de las plantaciones, la detección de incidencias y la recopilación sistemática de información, reduciendo al mismo tiempo la intervención manual continua del operador.



Figura 2.4. Dron utilizado en agricultura. Fuente: [18].

Apoyo en emergencias y búsqueda de objetivos: En escenarios de emergencia, catástrofe o difícil acceso, la navegación autónoma permite explorar áreas amplias, evitar obstáculos y centrar la misión en la localización de objetivos de interés, como personas, embarcaciones, vehículos o puntos críticos del terreno. Esta capacidad resulta valiosa en operaciones de apoyo en emergencias, donde el sistema debe reaccionar, adaptarse a cambios del entorno y mantener una búsqueda continua [21].

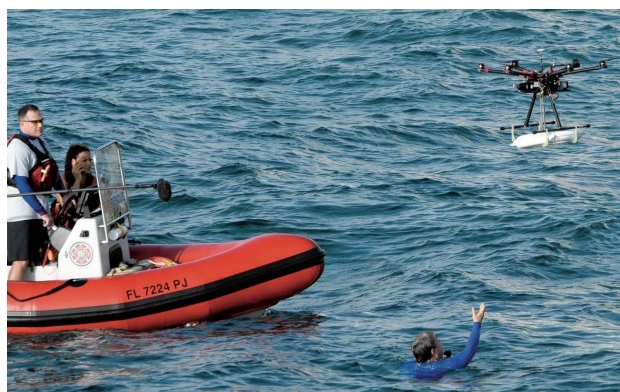


Figura 2.5. Dron utilizado en búsqueda y rescate Marítimo. Fuente: [20].

Transporte autónomo de mercancías: En entornos urbanos, industriales o logísticos [23], los UAV pueden convertirse en una alternativa eficiente para

el envío de pequeñas cargas, repuestos o material de primera necesidad. En este tipo de operaciones, la navegación autónoma permite reajustar la ruta ante restricciones del entorno, variaciones en el trayecto o condiciones externas no previstas, manteniendo al mismo tiempo un vuelo estable y seguro .



Figura 2.6. Dron utilizado para transporte de paquetes. Fuente: [22].

Operaciones militares y de defensa: Este es el caso de uso más próximo al enfoque de este Trabajo de Fin de Grado. En misiones de reconocimiento, vigilancia, defensa, interceptación [26, 25] y seguimiento de UAV enemigos, la capacidad de reaccionar ante amenazas dinámicas resulta fundamental. En estos escenarios, el sistema no solo debe desplazarse, sino también detectar, seguir y mantener el objetivo dentro del campo de visión, corrigiendo su trayectoria en tiempo real incluso ante maniobras evasivas.



Figura 2.7. Drones utilizados para operaciones militares. Fuente: [24].

En consecuencia, los métodos basados exclusivamente en pilotaje humano o en trayectorias rígidamente prefijadas resultan insuficientes cuando

el sistema debe desenvolverse en entornos no estructurados y frente a objetivos dinámicos. Además, este enfoque incrementa la dependencia de pilotos con un alto nivel de entrenamiento, capaces de mantener decisiones rápidas y precisas durante situaciones de elevada exigencia operativa. Por ello, este Trabajo de Fin de Grado se orienta hacia estrategias de percepción visual y control de navegación que permitan reducir esa dependencia, automatizar tareas críticas de seguimiento e intercepción y dotar al UAV cazador de una respuesta robusta, precisa y adaptable en tiempo real.

2.3. Métodos de Control de Navegación para UAVs

Para alcanzar este nivel de autonomía, el control del vehículo no puede entenderse como una acción aislada, sino como parte de un sistema integrado de Guiado, Navegación y Control, habitualmente denominado GNC (*Guidance, Navigation and Control*). Este marco organiza el comportamiento inteligente de la aeronave en tres niveles jerárquicos y complementarios: el guiado determina la estrategia de la misión y la trayectoria a seguir; la navegación estima el estado del UAV y su posición relativa respecto al entorno; y el control transforma esta información en comandos específicos de actitud y empuje. En conjunto, el sistema GNC permite cerrar el lazo entre la percepción sensorial y la respuesta dinámica del dron, un factor crítico cuando el escenario de operación cambia en tiempo real y el objetivo se mueve de forma continua .

Dentro de este esquema, el control de navegación actúa como el núcleo de decisión táctica. Su función principal es procesar la información proveniente del módulo de navegación para generar comandos de movimiento que permitan cumplir objetivos complejos, como la persecución de una presa. En este tipo de misiones, el éxito no depende únicamente de una detección correcta, sino de la capacidad del algoritmo para orquestar un movimiento fluido que mantenga al objetivo dentro del campo de visión sin comprometer la estabilidad del vuelo.

Dada la complejidad de este desafío, la literatura científica ha evolucionado hacia diversos paradigmas de control, diferenciados principalmente por la fuente de los datos y la arquitectura del procesamiento 2.8. Estos enfoques se pueden clasificar en tres grandes categorías : los métodos basados en localización global, los enfoques fundamentados en aprendizaje profundo (*Deep Learning*) y los sistemas de control basado en imagen (*Visual Servoing*). Cada una de estas metodologías presenta un compromiso distinto entre precisión, coste computacional y robustez, factores que determinan su idoneidad para operar en sistemas empotrados de recursos limitados. A continuación, se describen los fundamentos y aplicaciones de estos paradigmas

para situar las soluciones propuestas en la literatura y preparar el marco metodológico del trabajo.

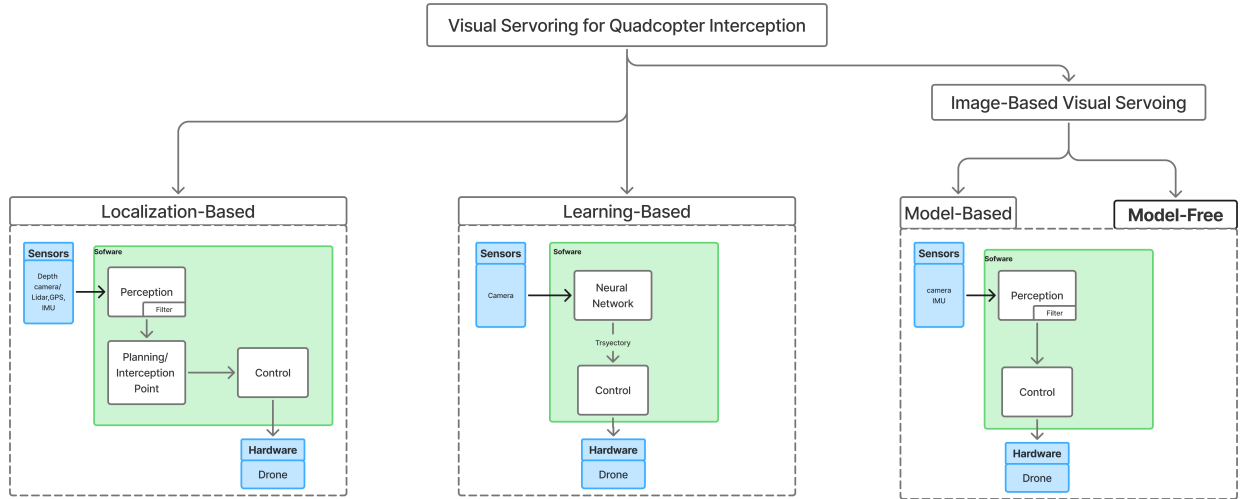


Figura 2.8. Diferencias entre los tipos de algoritmos de visual Servoing(Elaboración propia).

2.3.1. Tipos de control de navegación

Los **métodos basados en localización**, encuadrados habitualmente en el paradigma *Position-Based Visual Servoing* (PBVS), tienen como objetivo reconstruir la posición y la orientación del blanco en el espacio cartesiano a partir de la información visual [28]. Esta estimación tridimensional permite definir trayectorias de intercepción con un mayor grado de precisión, algo especialmente útil en misiones de seguimiento e intercepción con cuadricópteros. Dentro de esta familia, algunos trabajos recurren a controladores predictivos no lineales que optimizan la acción de control en un horizonte temporal, teniendo en cuenta tanto la evolución prevista del objetivo como las restricciones dinámicas del UAV. Otros enfoques emplean lazos de control sobre velocidades angulares para hacer coincidir la línea de visión con el vector de velocidad del vehículo, o modelan la trayectoria del blanco en tres dimensiones para escoger puntos de intercepción más convenientes. De igual modo, también se han propuesto arquitecturas de doble lazo PID que aprovechan los estados tridimensionales estimados del objetivo para seguir blancos terrestres o aéreos[34].

En definitiva, la principal fortaleza de PBVS radica en su capacidad para prever el movimiento del objetivo y determinar puntos de intercepción.

No obstante, su rendimiento queda condicionado por una estimación fiable de la profundidad y por una calibración rigurosa de la cámara. Asimismo, una parte importante de los trabajos revisados complementa la información visual con sensores adicionales, como radares o cámaras estéreo y de profundidad, con el fin de mejorar la reconstrucción espacial y aumentar la robustez del sistema.

Los **métodos basados en aprendizaje** delegan una parte importante del control en modelos entrenados con datos, normalmente redes neuronales capaces de inferir comandos de navegación de forma directa a partir de la información visual. Dentro de esta familia destacan enfoques de aprendizaje supervisado y, especialmente, técnicas de aprendizaje por refuerzo, que han mostrado una notable capacidad de adaptación frente al ruido sensorial y ante entornos dinámicos o poco estructurados. Esta flexibilidad resulta especialmente atractiva en tareas de interceptación, donde el dron debe reaccionar en tiempo real a cambios imprevistos de la escena o de la trayectoria del objetivo [37, 38]. No obstante, su aplicación práctica sigue presentando dificultades relevantes: requieren grandes volúmenes de datos y tiempos de entrenamiento elevados, suelen quedar condicionados por los escenarios utilizados durante el aprendizaje y continúan arrastrando problemas importantes de transferencia entre simulación y mundo real. A ello se suma la cuestión de la generalización, ya que un modelo entrenado en condiciones concretas puede perder fiabilidad cuando cambian el entorno, la dinámica del vehículo o las perturbaciones externas [39].

Por su parte, los **métodos basados en imagen**, agrupados bajo el paradigma *Image-Based Visual Servoing* (IBVS), utilizan directamente el error medido en el plano imagen para generar la acción de control. En lugar de reconstruir por completo la geometría tridimensional del objetivo, estos enfoques trabajan con características visuales como el desplazamiento del blanco respecto al centro de la imagen o la variación de su tamaño aparente. Esta decisión reduce la dependencia respecto a estimaciones geométricas completas y suele favorecer soluciones más ligeras y mejor adaptadas a plataformas embarcadas con recursos limitados [28, 29, 31].

2.3.2. Conceptos de Visual Servoing (IBVS)

Dentro de IBVS pueden distinguirse, a su vez, dos enfoques. El primero corresponde a los métodos basados en modelo, que incorporan de forma explícita la dinámica del cuadricóptero y la cinemática visual en el diseño del controlador. Estos métodos permiten respuestas más predictivas, un seguimiento más preciso y, en algunos casos, garantías formales de estabilidad, pero a cambio requieren un modelado más detallado del sistema y un mayor esfuerzo de ajuste [58, 29, 31]. El segundo enfoque es el de los métodos libres

de modelo, en los que el controlador abstrae gran parte de la dinámica interna del UAV y delega la estabilización básica en el autopiloto o en los lazos de bajo nivel ya disponibles. Esta aproximación reduce la carga computacional y simplifica la integración práctica, aunque puede ofrecer una menor capacidad de respuesta ante maniobras extremadamente agresivas o escenarios muy exigentes [32, 33, 35].

Por tanto, la elección entre PBVS, enfoques basados en aprendizaje e IBVS no depende únicamente de cuál resulte más sofisticado desde un punto de vista teórico, sino de cuál se adapta mejor al contexto de aplicación. PBVS aporta una representación espacial rica, pero exige una percepción 3D precisa; los métodos basados en aprendizaje ofrecen flexibilidad, aunque todavía presentan dificultades importantes de entrenamiento y transferencia al entorno real; e IBVS proporciona una relación especialmente favorable entre simplicidad, latencia y viabilidad en tiempo real. En problemas donde la percepción ya suministra detecciones en el plano imagen, los enfoques IBVS libres de modelo constituyen una opción especialmente coherente, ya que permiten conectar de forma directa la localización visual del objetivo con la generación de consignas de control sin introducir una complejidad geométrica o computacional excesiva. Esta observación explica que buena parte de la literatura combine percepción visual, estimación ligera y control reactivo, y sirve además como punto de partida para la metodología desarrollada posteriormente.

2.4. Deep Learning

El *Deep Learning* es una rama del aprendizaje automático basada en redes neuronales artificiales con múltiples capas, capaces de aprender representaciones jerárquicas directamente a partir de los datos. A diferencia de otros enfoques más tradicionales, reduce la necesidad de diseñar manualmente las características de entrada, ya que el propio modelo ajusta internamente niveles sucesivos de abstracción durante el proceso de entrenamiento [16].

El desarrollo de esta disciplina ha estado impulsado por la disponibilidad de grandes volúmenes de datos, el aumento de la capacidad de cómputo y la mejora de los métodos de entrenamiento. Gracias a ello, el *Deep Learning* se ha consolidado como un enfoque general para abordar tareas de clasificación, regresión, predicción y generación de datos en contextos muy diversos. Esta flexibilidad explica su relevancia dentro del aprendizaje automático actual y justifica su presencia como base conceptual en numerosos sistemas inteligentes [16].

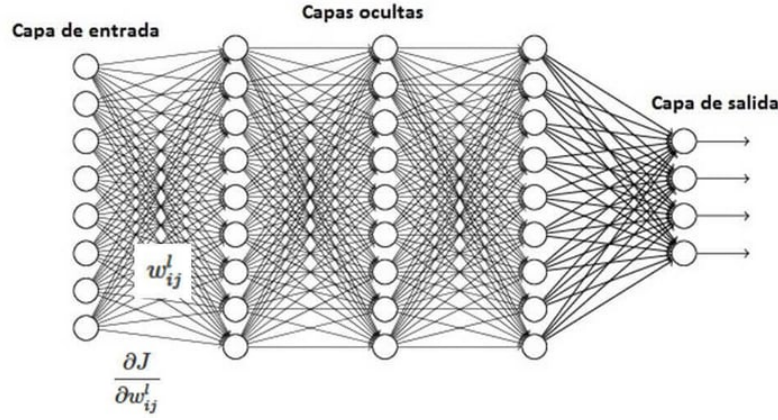


Figura 2.9. Esquema general de una red neuronal multicapa. Fuente: [11].

2.5. Modelos de regresión

En aprendizaje automático, un modelo de regresión es aquel que estima variables continuas a partir de una entrada dada. A diferencia de los modelos de clasificación, cuyo objetivo es asignar una etiqueta discreta, la regresión predice magnitudes numéricas, como una posición, una velocidad, una distancia, una temperatura o cualquier otra cantidad medible [16].

Desde un punto de vista funcional, un modelo de regresión recibe un conjunto de variables de entrada y aprende una función que relaciona esas entradas con una salida continua. Durante el entrenamiento, el modelo ajusta sus parámetros para reducir el error entre la predicción y el valor real observado. Una forma general de expresar este comportamiento es la siguiente:

$$\hat{y} = f_{\theta}(x) \quad (2.1)$$

donde x representa la información de entrada, f_{θ} el modelo parametrizado y $\hat{y}$ el valor continuo estimado. Una vez entrenado, el modelo puede utilizar la relación aprendida para predecir valores sobre datos no vistos previamente [16].

Por tanto, los modelos de regresión sirven para estimar magnitudes de interés, cuantificar relaciones entre variables y apoyar la toma de decisiones en problemas donde la salida no es una clase, sino un valor numérico. Se emplean, por ejemplo, para prever tendencias, estimar estados físicos, calcular distancias o aproximar parámetros que no se observan de forma directa.

En este contexto, la regresión aporta la información geométrica necesaria para localizar el objeto en la imagen, lo que enlaza de forma natural con la detección de objetos, tratada en el apartado siguiente.

2.6. Visión por Computador

La visión por computador, también conocida como visión artificial o visión computarizada, es una rama de la inteligencia artificial que agrupa técnicas y herramientas orientadas a adquirir, procesar e interpretar información visual procedente del entorno. A partir de imágenes o secuencias de vídeo, estos sistemas pueden extraer características relevantes que facilitan la detección, la clasificación y el análisis de objetos, escenas y movimientos.

Este campo integra conocimientos de procesamiento digital de imágenes, estadística, aprendizaje automático y robótica con el propósito de desarrollar sistemas capaces de comprender el contenido visual y actuar a partir de él, de forma autónoma o asistida. En este sentido, la visión por computador busca dotar a las máquinas de capacidades de percepción útiles para interpretar su entorno y apoyar procesos posteriores de decisión y control.

Sus orígenes suelen situarse en la década de 1960, cuando comenzaron a explorarse métodos para que los ordenadores reconocieran formas sencillas en imágenes bidimensionales. En aquella etapa inicial, las aplicaciones se centraban en tareas relativamente básicas, como la detección de bordes o el reconocimiento de patrones geométricos, debido tanto a las limitaciones del hardware disponible como a la escasez de datos con los que entrenar y validar los modelos.

En las últimas décadas, el incremento de la capacidad de cómputo, la disponibilidad de grandes conjuntos de datos y el desarrollo de arquitecturas basadas en redes neuronales convolucionales (CNN) han impulsado notablemente la evolución del área. Gracias a ello, hoy es posible abordar tareas complejas como la detección de objetos en tiempo real, la segmentación semántica, el reconocimiento facial o el seguimiento automático, y han surgido detectores especialmente influyentes como Faster R-CNN y la familia YOLO [41].

Entre sus aplicaciones destacan la conducción autónoma, la videovigilancia, la robótica, la inspección automática y la interacción persona-máquina. En el contexto de este trabajo, la visión por computador se empleará como base perceptiva para la detección y el seguimiento visual del dron objetivo, lo que enlaza de forma natural con la detección de objetos tratada en el apartado siguiente .

2.6.1. Paradigmas de detección de objetos

El objetivo fundamental de la detección de objetos consiste en localizar y clasificar regiones de interés dentro de una imagen, normalmente representadas mediante cajas delimitadoras. La detección debe resolver simultáneamente qué objetos aparecen y dónde se encuentran. Por ello, se trata de un problema más exigente desde el punto de vista computacional y metodológico, ya que combina percepción semántica y localización espacial en una misma tarea.

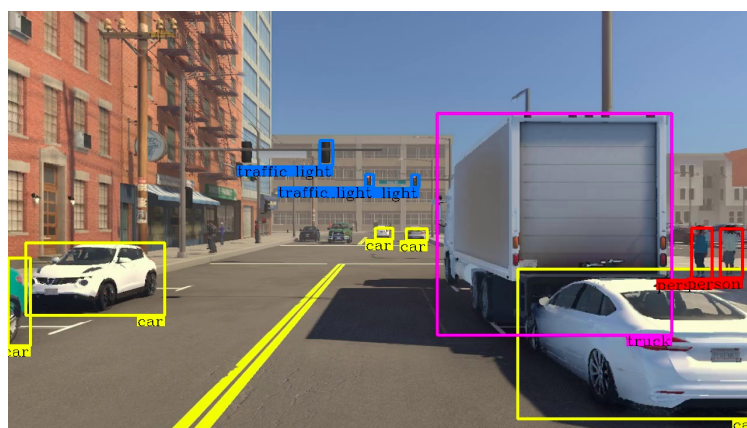


Figura 2.10. Ejemplo ilustrativo de detección de objetos mediante cajas delimitadoras. Fuente: [17].

Antes de la consolidación de los enfoques basados en redes profundas, una parte importante de los métodos de detección se apoyaba en estrategias de ventana deslizante. En ese marco, modelos como *Deformable Parts Model* (DPM) recorrían la imagen evaluando un gran número de subregiones candidatas, lo que permitía detectar objetos a distintas escalas y posiciones, pero a costa de una carga computacional muy elevada. Aunque estos enfoques marcaron una etapa importante en la evolución de la detección de objetos, su coste de inferencia y su limitada capacidad de adaptación a escenas complejas hicieron evidente la necesidad de arquitecturas más eficientes [43].

La transición hacia el *Deep Learning* se consolidó con la familia R-CNN, que introdujo un esquema de dos etapas. En su formulación original, R-CNN generaba primero un conjunto de regiones potencialmente relevantes mediante búsqueda selectiva, después extraía características con una red convolucional para cada una de esas regiones y, finalmente, realizaba la clasificación. Este planteamiento supuso un avance decisivo en precisión, pero seguía penalizado por el hecho de procesar cada región de manera separada y por depender de un mecanismo externo de propuesta de regiones. Faster R-CNN mejoró este pipeline al integrar la generación de propuestas dentro

de la propia red mediante una *Region Proposal Network* (RPN). Con ello, el sistema gana coherencia y velocidad respecto a R-CNN, aunque sigue siendo un detector de dos etapas y, por tanto, mantiene una latencia relativamente mayor que otras alternativas [41, 40].

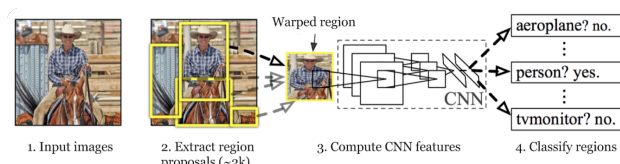


Figura 2.11. Esquema simplificado del funcionamiento de R-CNN, [41].

Frente a esta familia de detectores, los métodos de una sola etapa reformulan la detección como un problema unificado de predicción directa. En lugar de separar explícitamente la propuesta de regiones y la clasificación, una única red procesa la imagen completa y estima de forma simultánea las categorías, las puntuaciones de confianza y las coordenadas de las cajas delimitadoras. Esta simplificación reduce la complejidad del pipeline y favorece tiempos de inferencia mucho menores, una propiedad especialmente relevante en plataformas embarcadas y en escenarios dinámicos como la intercepción aire-aire, donde la frecuencia de actualización del detector resulta tan importante como su precisión.

2.6.2. YOLO

La familia YOLO (*You Only Look Once*) constituye el ejemplo más influyente de detector de una sola etapa basado en regresión unificada. Su aportación principal consiste en reformular la detección de objetos como un problema de predicción directa sobre la imagen completa, evitando la separación explícita entre propuesta de regiones y clasificación. De este modo, la red estima de forma conjunta la localización de las cajas delimitadoras y la probabilidad de las clases presentes en la escena.

Desde un punto de vista operativo, estos detectores combinan tres ideas fundamentales: una partición implícita de la escena en regiones de predicción, la estimación directa de cajas delimitadoras y puntuaciones de confianza mediante regresión, y una etapa final de depuración basada en métricas de solapamiento como la intersección sobre unión (IoU) y en procedimientos como *Non-Maximum Suppression* (NMS). Esta formulación permite generar detecciones con latencias reducidas y con una frecuencia de actualización compatible con aplicaciones de tiempo real.

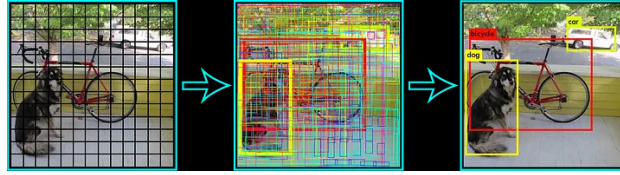


Figura 2.12. Esquema simplificado del funcionamiento de Yolo, [49].

La literatura reciente ha consolidado a YOLO como una de las familias más extendidas en visión embarcada gracias a su equilibrio entre precisión, velocidad y facilidad de despliegue. En problemas de detección aérea, seguimiento e intercepción, este compromiso resulta especialmente valioso porque la utilidad del detector depende no solo de su exactitud espacial, sino también de su capacidad para suministrar observaciones consistentes a alta frecuencia. Además, al predecir directamente magnitudes continuas asociadas a la localización del blanco, estos modelos proporcionan una base adecuada para etapas posteriores de seguimiento visual, estimación geométrica e inferencia de distancia relativa [48].

2.7. Geometría Proyectiva

Una vez presentadas las técnicas de percepción visual y detección de objetos, resulta necesario introducir el marco geométrico que permite interpretar esa información en términos espaciales y relacionarla con el movimiento del dron. En este contexto, la geometría proyectiva constituye la base teórica que conecta los puntos del espacio tridimensional con su representación en el plano imagen. En navegación visual, esta disciplina resulta esencial porque la cámara no proporciona directamente una posición 3D del objetivo, sino una proyección bidimensional condicionada por la perspectiva, la orientación de la cámara y sus parámetros internos [63, 65, 64]. Antes de introducir el modelo geométrico de la cámara, conviene partir de la cámara estenopeica o modelo *pinhole*, ya que constituye la aproximación más simple y utilizada para explicar cómo se forma una imagen.

2.7.1. Cámara estenopeica

La cámara estenopeica, también conocida como modelo *pinhole*, es una idealización geométrica utilizada para describir cómo se forma una imagen a partir de la luz procedente del entorno. En este modelo, los rayos luminosos atraviesan un único orificio y se proyectan sobre un plano imagen, lo que permite relacionar de forma sencilla el espacio tridimensional con su representación bidimensional.

Sus antecedentes históricos se asocian a la cámara oscura y a las primeras observaciones sobre la propagación rectilínea de la luz. desde la Antigüedad se estudiaron fenómenos de proyección similares, y con el tiempo estos principios dieron lugar a dispositivos estenopeicos cada vez más elaborados, primero con fines de observación y más adelante también con aplicaciones fotográficas y didácticas [60].

Desde un punto de vista funcional, su funcionamiento se basa en que solo una pequeña fracción de la luz procedente de cada punto de la escena atraviesa el orificio, lo que genera una imagen invertida sobre el plano de proyección. Así, el sistema no mide directamente la profundidad ni las coordenadas espaciales del entorno, sino su proyección en la imagen. Esta idea resulta fundamental para comprender cómo la posición, el tamaño aparente y la perspectiva de los objetos quedan codificados visualmente .

Tradicionalmente, la cámara estenopeica se construía de forma muy simple, mediante una caja opaca, un pequeño estenopo y una superficie fotosensible, como papel o película fotográfica, por lo que las exposiciones solían ser largas y el control de entrada de luz era manual. En la actualidad, ese mismo principio puede aplicarse también sobre dispositivos con sensores digitales, como los CCD, y sigue utilizándose con fines educativos, artísticos y experimentales. De este modo, puede compararse una versión histórica basada en captura analógica con variantes modernas apoyadas en tecnología digital [60].

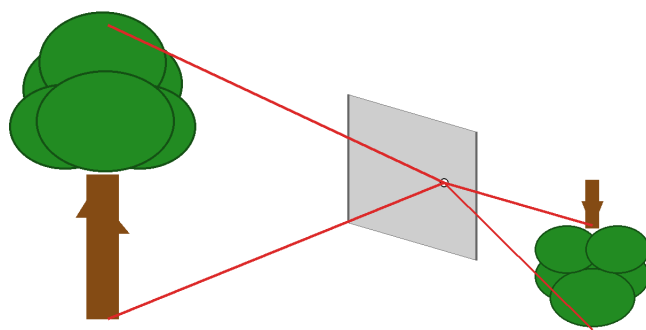


Figura 2.13. Funcionamiento simplificado de una cámara estenopeica. Fuente: [61].

2.7.2. Modelo geométrico de la cámara

A partir de esta idealización, la proyección de un punto del mundo puede representarse de forma compacta mediante la relación entre parámetros extrínsecos, parámetros intrínsecos y coordenadas del punto observado. En forma homogénea, esta idea suele resumirse como:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R|t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (2.2)$$

donde la matriz intrínseca recoge parámetros como la distancia focal y el punto principal, mientras que la transformación extrínseca describe la posición y orientación de la cámara respecto al entorno. Aunque se trata de un modelo idealizado, resulta suficientemente expresivo para describir la mayor parte de los fenómenos geométricos relevantes en visión por computador [59].

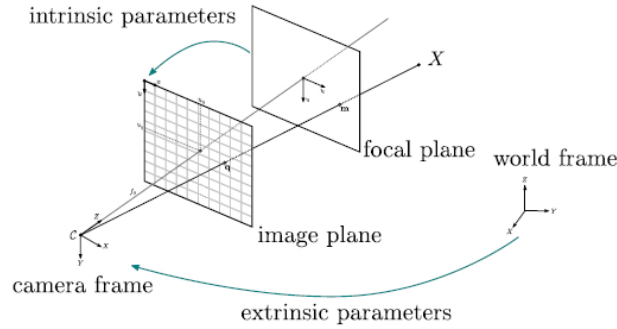


Figura 2.14. Esquema simplificado del modelo de cámara estenopeica. Fuente: [62].

2.7.3. Usos en navegación visual

En la literatura sobre *visual servoing* y seguimiento de UAV, la geometría proyectiva se emplea principalmente con tres objetivos. En primer lugar, permite calibrar el sistema de visión y caracterizar propiedades como el campo de visión, el punto principal o las distorsiones introducidas por la óptica. En segundo lugar, proporciona la base para relacionar variables medidas en píxeles con magnitudes útiles para estimación y control, como la dirección de visión, la escala aparente del objetivo o su posición relativa. En tercer lugar, sirve de soporte para compensar el efecto de la actitud del

vehículo y para diseñar controladores que actúan a partir del error visual observado [58, 29].

Por ello, incluso en enfoques IBVS que evitan una reconstrucción tridimensional completa, la geometría proyectiva sigue siendo una herramienta conceptual relevante. Más que un bloque aislado de reconstrucción 3D, actúa como un lenguaje común que conecta calibración, percepción y control dentro de los sistemas de navegación guiados por cámara [35].

2.8. Métodos de Control

En robótica aérea, el control puede definirse como el conjunto de algoritmos encargados de manipular las variables de entrada del sistema, como la velocidad de los motores o las consignas de actitud, para lograr que las variables de salida del vehículo, entre ellas su posición, velocidad y orientación, sigan un comportamiento deseado. Su papel es fundamental para mantener la estabilidad frente a perturbaciones externas y para garantizar que el UAV cumpla de forma autónoma los objetivos de la misión .

2.8.1. Sistemas de Lazo Abierto vs. Lazo Cerrado

La clasificación más básica de los sistemas de control depende de la existencia o no de retroalimentación entre la salida y la acción de control [68, 69].

- **Control de lazo abierto (*open-loop*)**. La señal de control se genera sin tener en cuenta el estado real de la salida. En un UAV, esto equivaldría a enviar una orden de movimiento sin verificar si el vehículo ha respondido realmente como se esperaba. Este enfoque resulta poco adecuado para navegación autónoma, ya que no corrige errores ni compensa perturbaciones como el viento o pequeñas variaciones dinámicas del sistema.
- **Control de lazo cerrado (*closed-loop*)**. El sistema mide continuamente la salida, la compara con una referencia deseada y calcula un error a partir de esa diferencia. A continuación, el controlador actualiza la acción de mando para reducir dicho error. Este es el paradigma predominante en navegación autónoma basada en visión, ya que permite cerrar el lazo entre la percepción visual y la respuesta dinámica del dron. En este contexto, la referencia puede asociarse, por ejemplo, a mantener el objetivo centrado en la imagen, mientras que la salida medida corresponde a la posición observada del blanco en el plano imagen.

2.8.2. Controlador PID

Dentro de los mecanismos de control por retroalimentación, el controlador PID (*Proportional-Integral-Derivative*) destaca como una de las soluciones clásicas más utilizadas por su sencillez, su bajo coste computacional y su eficacia práctica en numerosos sistemas dinámicos. Su idea central consiste en calcular la acción de control a partir de la evolución temporal del error entre una referencia deseada y la salida medida del sistema [68, 69].

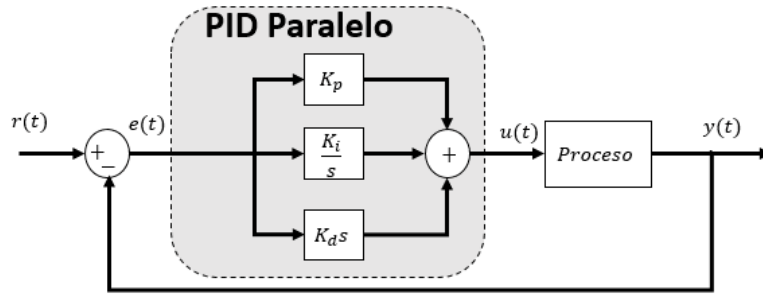


Figura 2.15. Esquema del funcionamiento de un PID., [67].

Si se define el error como:

$$e(t) = r(t) - y(t) \quad (2.3)$$

donde $r(t)$ es la referencia deseada y $y(t)$ es la salida observada, entonces la señal de control generada por un PID puede expresarse como:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.4)$$

Esta formulación combina tres contribuciones complementarias:

- **Término proporcional (K_p).** Proporciona una respuesta inmediata en función del error actual. Si el objetivo está alejado de la referencia, la acción de control aumenta de forma proporcional para corregir esa desviación con rapidez.
- **Término integral (K_i).** Acumula el error a lo largo del tiempo. Su función principal es eliminar el error residual o estático, de modo que el sistema no se quede permanentemente cerca de la referencia sin llegar a alcanzarla completamente.

- **Término derivativo (K_d).** Actúa sobre la velocidad de cambio del error. Su efecto puede interpretarse como un amortiguamiento que anticipa la tendencia del sistema y ayuda a reducir oscilaciones o respuestas demasiado bruscas.

Desde un punto de vista funcional, el término proporcional aporta rapidez de reacción, el integral mejora la precisión final y el derivativo contribuye a suavizar la dinámica transitoria. La combinación adecuada de estas tres acciones permite construir un controlador que responda con suficiente rapidez sin comprometer la estabilidad del sistema.

Por estas razones, el PID sigue siendo una elección frecuente en plataformas embebidas que deben compartir recursos con módulos de percepción e inferencia en tiempo real. Frente a enfoques de control más complejos, ofrece una implementación ligera y una integración sencilla con los lazos internos del autopiloto, lo que explica su amplia presencia en aplicaciones de seguimiento visual de UAV.

2.9. Estimación de Estado y Filtrado

En sistemas de navegación visual, las observaciones extraídas por el detector suelen estar afectadas por ruido, retardos y pérdidas puntuales. Por ello, la literatura incorpora con frecuencia técnicas de estimación y filtrado que aportan coherencia temporal a la información perceptiva antes de utilizarla en el lazo de control. Entre las soluciones más extendidas destacan el filtro de Kalman y los métodos de suavizado recursivo, como el EMA, por su equilibrio entre coste computacional y capacidad para atenuar fluctuaciones de la señal.

2.9.1. Origen y Evolución del Filtro de Kalman

El filtro de Kalman es un método de estimación recursiva formulado por Rudolf E. Kalman en 1960 para inferir el estado de sistemas dinámicos lineales a partir de medidas afectadas por ruido. Su interés radica en que combina la predicción proporcionada por un modelo dinámico con la información de los sensores, obteniendo estimaciones más estables y consistentes que las medidas directas por sí solas [52, 53, 54].

Con el tiempo, esta formulación se ha extendido a variantes no lineales, como el filtro de Kalman extendido (EKF), muy utilizadas en robótica móvil y aérea. En navegación visual con UAV, además, el filtrado suele integrarse con modelos dinámicos y estrategias predictivas para compensar retardos

de percepción, suavizar detecciones y mantener la coherencia temporal del seguimiento.

2.9.2. Fundamentos y Mecanismo Recursivo

Desde un punto de vista operativo, el filtro funciona de manera recursiva y en tiempo real. En cada instante combina una etapa de predicción, basada en el modelo del sistema, con una etapa de actualización, en la que incorpora la nueva medida disponible. Esta estructura lo hace especialmente apropiado para plataformas embebidas, donde interesa limitar el coste computacional y no almacenar todo el historial de observaciones.

El sistema se representa mediante un modelo de espacio de estados:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (2.5)$$

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k \quad (2.6)$$

En estas ecuaciones, $\mathbf{x}_k$ representa el estado del sistema, $\mathbf{z}_k$ la observación disponible, $\mathbf{F}_k$ la dinámica del modelo y $\mathbf{H}_k$ la relación entre el estado y la medida. Los términos $\mathbf{w}_k$ y $\mathbf{v}_k$ modelan, respectivamente, el ruido del proceso y el ruido de medida.

2.9.3. Aplicación en Persecución Visual y Alta Velocidad

En sistemas de persecución visual, el filtrado de Kalman aporta continuidad temporal a las detecciones, atenúa el ruido de la medida y permite anticipar a corto plazo el movimiento del objetivo. Estas propiedades resultan especialmente relevantes cuando la información visual llega con retardos, presenta oclusiones parciales o debe combinarse con la dinámica del interceptor dentro del lazo de control.

En la literatura reciente, este tipo de estimación se integra con estrategias de navegación visual para fusionar la información procedente de la cámara con el modelo dinámico del vehículo, lo que contribuye a mantener trayectorias de seguimiento más estables y a reducir el efecto de fluctuaciones rápidas en la percepción. De este modo, el filtrado no sustituye al controlador, sino que mejora la calidad de la señal sobre la que este actúa.

Entre sus aportaciones más relevantes en este contexto pueden destacarse las siguientes:

- **Continuidad temporal:** ayuda a mantener una estimación coherente del objetivo incluso ante pérdidas puntuales de detección.
- **Fusión entre modelo y medida:** combina la información visual con la predicción dinámica del sistema para reducir la sensibilidad al ruido.
- **Predicción a corto plazo:** facilita anticipar la evolución inmediata del blanco, algo especialmente útil en maniobras rápidas o agresivas.

2.9.4. Filtro EMA

El filtro EMA (*Exponential Moving Average*), también denominado suavizado exponencial, es un método recursivo de suavizado utilizado para reducir las fluctuaciones rápidas de una señal sin necesidad de almacenar un gran número de muestras previas. Su interés principal reside en que requiere muy poca memoria y un coste computacional muy bajo, por lo que resulta especialmente adecuado para sistemas embebidos y aplicaciones de control en tiempo real [55, 56, 57].

Desde un punto de vista funcional, el EMA actúa como un filtro paso bajo: atenúa las variaciones de alta frecuencia asociadas al ruido y conserva la tendencia general de la señal. Por este motivo, se utiliza de forma habitual cuando se desea reducir la sensibilidad de una señal frente a pequeñas oscilaciones entre muestras consecutivas.

La formulación discreta del filtro EMA es la siguiente:

$$\hat{x}_k = \alpha x_k + (1 - \alpha)\hat{x}_{k-1} \quad (2.7)$$

donde x_k representa la medida actual, $\hat{x}_k$ la salida filtrada y $\hat{x}_{k-1}$ el valor filtrado del instante anterior. El parámetro α , con $0 < \alpha \leq 1$, determina el peso relativo entre la muestra nueva y el historial previo.

Esta expresión también puede escribirse como:

$$\hat{x}_k = \hat{x}_{k-1} + \alpha (x_k - \hat{x}_{k-1}) \quad (2.8)$$

Esta segunda forma resulta útil para interpretar el funcionamiento del filtro: la nueva salida se obtiene corrigiendo el valor filtrado anterior en función de la diferencia entre la medida actual y la estimación previa. Si α

toma valores pequeños, el suavizado es mayor y la respuesta del filtro es más lenta; si α se aproxima a 1, la salida sigue más de cerca la medida y el efecto de suavizado disminuye .

La Figura 2.16 ilustra este comportamiento sobre una señal ruidosa. En ella puede apreciarse cómo la salida filtrada atenúa las oscilaciones rápidas de la señal original y conserva su tendencia global.

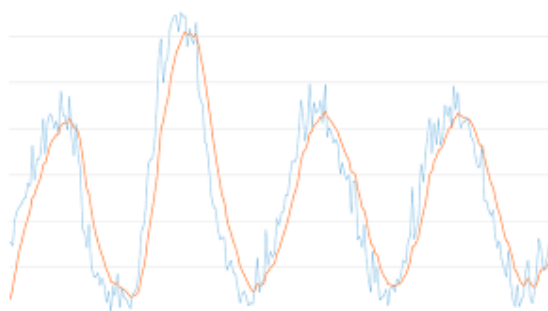


Figura 2.16. Efecto del filtro EMA sobre una señal ruidosa. Fuente: [57].

En consecuencia, el EMA constituye una técnica de suavizado sencilla, ligera y eficiente para señales discretas. Su bajo coste computacional y su formulación recursiva hacen que resulte especialmente útil en aplicaciones donde se requiere atenuar ruido de alta frecuencia sin introducir una complejidad elevada.

2.10. Stack de Software y Hardware

La sofisticación de los sistemas robóticos modernos, especialmente en el ámbito de las aeronaves no tripuladas que operan en entornos dinámicos, exige infraestructuras de software que permitan la ejecución paralela de procesos heterogéneos. En este contexto, resulta imperativo emplear marcos de trabajo que actúen como capas de abstracción o middleware, garantizando la interoperabilidad entre los diferentes módulos de percepción, planificación y control mediante intercambios de datos deterministas y eficientes. Estas plataformas aseguran una comunicación estandarizada entre procesos, facilitando la integración de algoritmos complejos de navegación autónoma.

2.10.1. Comunicación Distribuida en Robótica

Data Distribution Service (DDS)

El estándar Data Distribution Service se define como un middleware orientado a datos, regulado por la Object Management Group, y diseñado específicamente para entornos donde la fiabilidad y el determinismo son requisitos de misión crítica. Su naturaleza descentralizada, basada en un modelo de publicación y suscripción, elimina la dependencia de un servidor central, lo que potencia la resiliencia del sistema ante fallos puntuales de red o de nodos.

La característica más distintiva de esta tecnología es su capacidad para gestionar políticas de Calidad de Servicio (QoS). Estos parámetros permiten un control exhaustivo sobre la latencia, la prioridad de los mensajes y la persistencia de la información, adaptándose a las necesidades temporales estrictas de aplicaciones aeroespaciales y de vehículos autónomos. No obstante, su principal desventaja es su complejidad: requiere una configuración meticulosa de las políticas QoS y un conocimiento técnico avanzado, lo que puede dificultar su adopción en fases tempranas de desarrollo o en equipos con recursos limitados.

Message Queuing Telemetry Transport (MQTT)

Como contraparte, el protocolo MQTT se presenta como una solución optimizada para la transferencia de mensajes en redes con ancho de banda limitado y recursos computacionales restringidos, condiciones habituales en el ecosistema del Internet de las Cosas (IoT). A diferencia de arquitecturas peer-to-peer, MQTT se apoya en un broker centralizado que gestiona el tráfico entre publicadores y suscriptores, simplificando la topología de la red.

Si bien este protocolo ofrece diferentes niveles de fiabilidad en la entrega de datos, su diseño original no contempla los requisitos de sincronización y baja latencia necesarios para el control de vuelo en tiempo real. Por esta razón, su aplicación en el desarrollo de UAVs suele quedar restringida a tareas complementarias de telemetría o interfaces de monitorización remota, resultando insuficiente para gobernar los lazos de control cerrados que exigen algoritmos de interceptación o seguimiento visual.

Robot Operating System (ROS)

En el panorama actual de la robótica, ROS se ha consolidado como el ecosistema de desarrollo más versátil, proporcionando una capa de abstrac-

ción que convierte un sistema operativo convencional en un entorno robótico distribuido. Mediante el uso de nodos, topics y services, esta plataforma permite la creación de sistemas modulares donde la percepción visual y las leyes de control pueden interactuar de forma transparente.

El valor fundamental de este framework reside en su vasto repositorio de herramientas, que abarca desde la simulación en entornos de alta fidelidad como Gazebo hasta la visualización de estados internos y la gestión de sensores complejos. La transición hacia versiones más modernas ha permitido que el transporte de datos se apoye nativamente en el estándar DDS, unificando la flexibilidad del desarrollo modular con la robustez de las comunicaciones industriales. Para proyectos de navegación autónoma, esta arquitectura facilita la integración de nodos de gnc y proyección de imagen, permitiendo que el sistema responda a las exigencias de tiempo real impuestas por la dinámica de vuelo.

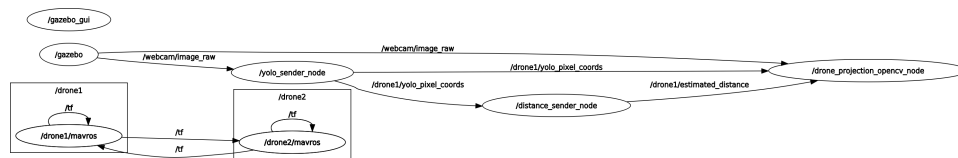


Figura 2.17. Ejemplo de conexión de Nodos durante ejecución (Imagen de Elaboración Propia)

2.10.2. Computación Distribuida Seleccionada

La implementación del sistema de navegación autónoma para la interceptación de aeronaves no tripuladas requiere de una infraestructura de software capaz de gestionar procesos concurrentes con un alto grado de desacoplamiento. En este sentido, se ha seleccionado el paradigma de Robot Operating System (ROS) como el middleware central, integrando componentes críticos como MAVROS para la interfaz con el hardware de vuelo.

La decisión de emplear ROS frente a otras alternativas técnicas se sustenta en la versatilidad de su ecosistema y la modularidad de su arquitectura. Si bien soluciones como Data Distribution Service (DDS) ofrecen una gestión granular de las políticas de Calidad de Servicio (QoS) y un control estricto sobre el determinismo de la red, su implementación exige una especialización técnica que puede ralentizar las fases iniciales de prototipado. No obstante, la evolución hacia ROS 2 permite heredar las garantías de comunicación de DDS de forma subyacente, manteniendo una interfaz de programación de alto nivel que simplifica la integración de módulos complejos de percepción y control.

Esta arquitectura modular resulta fundamental para la naturaleza del proyecto, permitiendo que el nodo de proyección geométrica pueda intercambiar información de forma eficiente con el bucle de control de guiado. A diferencia de protocolos como MQTT, que destacan por su ligereza en aplicaciones de telemetría y entornos de Internet de las Cosas (IoT), estos carecen de las capacidades necesarias para el control de actuadores en tiempo real y la sincronización de estados cinemáticos exigida en maniobras de persecución.

El ecosistema de ROS proporciona, además, una serie de bibliotecas y herramientas de grabación de datos que agilizan el ciclo de validación experimental. Esta flexibilidad facilita el escalado del sistema, permitiendo la incorporación de nuevas capacidades de visión artificial o técnicas de filtrado estocástico sin comprometer la cohesión del diseño original. ROS se presenta como la opción más completa y robusta para implementar un sistema de navegación vectorial distribuido, garantizando tanto la eficiencia computacional como la cohesión arquitectónica del conjunto.

ROS como base de la arquitectura distribuida

La arquitectura distribuida desarrollada en este proyecto se articula sobre Robot Operating System (ROS), elegido por su capacidad para coordinar múltiples procesos concurrentes dentro de un esquema modular y escalable. Más que un sistema operativo en sentido estricto, ROS actúa como un middleware robótico que establece una capa de abstracción entre el software de alto nivel y el hardware. Gracias a ello, facilita la comunicación entre módulos, la gestión de nodos, la supervisión del sistema y el desarrollo integrado de aplicaciones, cualidades especialmente relevantes en un contexto de navegación autónoma.

La estructura fundamental de ROS se organiza en torno al concepto de nodo, que constituye la unidad de ejecución lógica e independiente dentro del ecosistema del robot. Estas entidades funcionan como procesos autónomos que pueden ser desarrollados en diversos lenguajes de programación, aportando una gran modularidad al sistema completo. La transferencia de datos entre estos módulos se gestiona mediante tópicos, que actúan como canales de comunicación asíncrona bajo un esquema de publicación y suscripción.

Además de los tópicos, ROS dispone de mecanismos de comunicación complementarios, entre los que destacan los servicios y las acciones. Los servicios resultan apropiados para intercambios puntuales de tipo petición-respuesta entre nodos, por ejemplo cuando se requiere consultar el estado actual del UAV o recuperar la configuración de un módulo concreto. Las acciones, en cambio, están orientadas a tareas de ejecución prolongada que

necesitan retroalimentación continua, seguimiento del progreso o posibilidad de cancelación, como ocurre en el seguimiento de trayectorias con validación dinámica. Aun así, en este proyecto se ha priorizado una arquitectura basada exclusivamente en tópicos, ya que ofrece un comportamiento más adecuado para flujos de datos continuos y para las exigencias propias de la navegación y el control en tiempo real.



Figura 2.18. Robotics Operating System. Fuente: [2].

Comandos Básicos de ROS

El manejo de la línea de comandos constituye una parte esencial del trabajo con ROS Noetic, ya que permite poner en marcha el sistema, supervisar su estado y depurar incidencias durante la integración de los distintos nodos. A lo largo del desarrollo de este proyecto se han utilizado de forma recurrente varios comandos básicos, cada uno con una función concreta dentro del ecosistema ROS:

- **roscore.** Inicia el maestro de ROS y los servicios de comunicación fundamentales del sistema, por lo que su ejecución es imprescindible para que el resto de nodos pueda registrarse e intercambiar información.
- **roslaunch nombre_paquete nombre_nodo.** Permite ejecutar directamente un nodo específico contenido en un paquete previamente compilado, algo especialmente útil durante tareas de prueba y depuración aislada.
- **roslaunch nombre_paquete archivo.launch.** Se emplea para arrancar varios nodos de forma coordinada mediante archivos `.launch`, lo que facilita la inicialización de configuraciones más complejas.
- **rostopic list.** Muestra el conjunto de nodos activos registrados en el maestro, ofreciendo una visión rápida del estado del sistema en ejecución.
- **rostopic list.** Enumera los tópicos disponibles en ese momento, permitiendo identificar los canales de comunicación activos entre nodos.

- `rostopic echo nombre_topico`. Hace posible visualizar en tiempo real el contenido de los mensajes publicados en un t3pico determinado, lo que resulta de gran utilidad para verificar datos y detectar anomal3as en la comunicaci3n.
- `rosparam list`. Permite consultar el servidor de par3metros de ROS y revisar las variables de configuraci3n cargadas en el sistema.

MavROS como puente entre ROS y el sistema UAV

MAVROS constituye la capa de integraci3n que permite enlazar el ecosistema ROS con el protocolo MAVLink, ampliamente utilizado por autopilotos como PX4 y ArduPilot. Su papel dentro de la arquitectura no se limita a una simple pasarela de datos: actúa como un mecanismo de traducci3n entre los mensajes y servicios propios de ROS y las3rdenes, estados y par3metros que entiende el controlador de vuelo. Gracias a esta intermediaci3n, los m3dulos de percepci3n, estimaci3n y control pueden operar sobre una interfaz homog3nea sin depender directamente de los detalles internos del autopiloto.

Desde el punto de vista funcional, MAVROS habilita un canal bidireccional de comunicaci3n entre los nodos desarrollados en ROS y el UAV, ya sea en simulaci3n o sobre una plataforma real. A trav3s de este canal es posible recibir telemetr3a relacionada con la posici3n, la velocidad, la actitud, el estado de vuelo o el modo de operaci3n del veh3culo, al tiempo que se pueden enviar consignas de movimiento, comandos de misi3n y ajustes sobre determinados par3metros del sistema. Esta capacidad resulta especialmente valiosa en aplicaciones de navegaci3n aut3noma, donde el lazo de control debe alimentarse continuamente con informaci3n del estado del veh3culo y responder en tiempo real mediante nuevas referencias.

En el contexto de este proyecto, MAVROS se emplea como elemento de uni3n entre la l3gica de navegaci3n visual y el sistema de vuelo del UAV. Por un lado, permite que los algoritmos de seguimiento reciban de forma estructurada los datos necesarios para estimar el comportamiento del veh3culo. Por otro, hace posible que las trayectorias o correcciones generadas por el sistema propuesto se transmitan nuevamente al autopiloto utilizando una interfaz estandarizada. Esta separaci3n entre la l3gica de alto nivel y el hardware concreto aporta una ventaja importante: favorece la reutilizaci3n del software y reduce la dependencia de una plataforma a3rea espec3fica.

Otro aspecto relevante de MAVROS es su integraci3n con el sistema de transformaciones espaciales de ROS, gestionado mediante `tf`. Esto facilita la coordinaci3n entre distintos marcos de referencia, como `map`, `odom` o `base_link`, algo imprescindible para interpretar correctamente la informa-

ción de navegación y relacionar las observaciones visuales con el movimiento real del vehículo. No obstante, esta misma gestión de referencias puede convertirse en una fuente de complejidad cuando se trabaja con varios UAV de manera simultánea. En etapas tempranas del desarrollo, uno de los retos principales consistió precisamente en evitar interferencias y superposiciones entre los datos publicados por ambos sistemas, especialmente cuando compartían marcos de referencia con nombres comunes dentro del entorno de simulación.

En conjunto, MAVROS no solo simplifica la comunicación con el autopiloto, sino que también proporciona una base robusta para desacoplar percepción, control y ejecución de vuelo dentro de una arquitectura distribuida. Por ello, se convierte en una herramienta esencial para trasladar la lógica desarrollada en ROS a un sistema aéreo operativo, manteniendo al mismo tiempo modularidad, escalabilidad y portabilidad entre escenarios simulados y reales.

Defensa de la Elección

La arquitectura distribuida seleccionada presenta una serie de ventajas que justifican su elección dentro de este proyecto:

- **Modularidad.** La división del sistema en nodos independientes facilita la implementación progresiva de cada bloque, así como su mantenimiento y depuración.
- **Escalabilidad.** La arquitectura permite incorporar nuevos sensores, módulos de control o incluso múltiples UAV sin alterar de forma significativa el funcionamiento general del sistema.
- **Portabilidad.** El uso de estándares consolidados como ROS y MAVLink hace posible reutilizar la lógica desarrollada tanto en simulación como en plataformas reales con cambios mínimos.
- **Flexibilidad.** La integración de MAVROS con el ecosistema ROS permite trabajar con diferentes controladores de vuelo y configuraciones de vehículo de manera homogénea, tanto en pruebas simuladas como en escenarios reales.

En conclusión, estas ventajas permiten afirmar que la arquitectura adoptada no solo responde de manera adecuada a las exigencias técnicas del proyecto, sino que también favorece un desarrollo más ordenado, comprensible y sostenible en el tiempo. La combinación de ROS, MAVROS y herramientas de supervisión como RViz aporta un entorno de trabajo coherente para diseñar, observar y validar el comportamiento del sistema, lo que refuerza

la idoneidad de esta elección tanto en simulación como en futuras aplicaciones sobre plataformas reales. Asimismo, esta arquitectura deja preparado el sistema para futuras extensiones, entre ellas la integración de sensores más avanzados, nuevos algoritmos de percepción y estrategias de cooperación entre múltiples UAV en entornos distribuidos.

2.10.3. Simulación

La simulación ocupa un papel central en el desarrollo de sistemas robóticos autónomos, especialmente cuando se trabaja con vehículos aéreos no tripulados. Antes de trasladar un algoritmo a una plataforma física, resulta necesario disponer de un entorno controlado que permita estudiar con rigor el comportamiento dinámico del vehículo, la estabilidad del controlador y la respuesta del sistema ante perturbaciones o cambios en las condiciones de operación. En este sentido, el entorno simulado constituye una fase intermedia esencial entre el diseño teórico y la validación sobre hardware real.

Los simuladores tridimensionales ofrecen, además, la posibilidad de reproducir con suficiente realismo diversos elementos del entorno físico, como la gravedad, las colisiones, la respuesta cinemática y ciertas aproximaciones de la dinámica aerodinámica, junto con la emulación de sensores virtuales. Esto permite repetir ensayos en condiciones equivalentes, ajustar parámetros de forma iterativa y someter la solución a un proceso de prueba más amplio, seguro y económicamente asumible. Gracias a ello, se reducen de manera notable los riesgos sobre la plataforma física y se acelera la depuración del sistema antes de su despliegue en escenarios reales.

AirSim

AirSim es un simulador concebido específicamente para vehículos autónomos, desarrollado por Microsoft sobre el motor gráfico Unreal Engine. Entre sus principales fortalezas destaca la elevada calidad visual de los escenarios que genera, así como la posibilidad de emular sensores habituales en plataformas aéreas, como cámaras RGB, LIDAR, IMU o GPS. Por ello, suele considerarse una alternativa especialmente interesante en trabajos centrados en visión por computador, aprendizaje por refuerzo o navegación guiada principalmente por percepción.

No obstante, en el marco de este proyecto presenta algunas limitaciones relevantes. Aunque puede conectarse con ROS, su integración no resulta tan directa ni tan madura como la de otras plataformas más orientadas a la robótica experimental, y con frecuencia requiere componentes adicionales para comunicarse con fluidez con autopilotos como PX4 o ArduPilot. A ello se suma la exigencia computacional asociada al uso de Unreal Engine, que

puede dificultar la ejecución simultánea de varios UAV y hacer menos ágil el proceso de prueba.

Webots

Webots, desarrollado por Cyberbotics, constituye una solución de código abierto ampliamente utilizada en docencia e investigación. Ofrece una interfaz accesible, compatibilidad con distintos lenguajes de programación y soporte para una variedad amplia de sensores y actuadores. Además, dispone de integración con ROS, especialmente en entornos vinculados con ROS 2, lo que lo convierte en una plataforma versátil para numerosas tareas de robótica general.

Sin embargo, su idoneidad disminuye cuando se pretende reproducir con mayor fidelidad el comportamiento de un sistema UAV completo gobernado por autopilotos reales. La conexión con plataformas como PX4 o ArduPilot no alcanza el mismo grado de solidez que en otras alternativas, y su motor físico, aun siendo suficiente para muchos contextos, no siempre proporciona la precisión dinámica necesaria para analizar maniobras aéreas exigentes, control fino de trayectoria o escenarios multivehículo con un nivel más alto de realismo.

Gazebo

Gazebo se ha consolidado como uno de los simuladores más empleados en robótica por su equilibrio entre realismo físico, flexibilidad y facilidad de integración con el ecosistema ROS. Su arquitectura basada en plugins, la posibilidad de construir entornos modificables y su compatibilidad con múltiples sensores y modelos hacen de esta plataforma una opción especialmente adecuada para ensayar algoritmos de navegación, percepción y control en condiciones reproducibles.

En proyectos centrados en UAV, esta combinación de fidelidad e interoperabilidad resulta especialmente valiosa, ya que permite conectar de forma natural los nodos de control con los modelos del vehículo, automatizar experimentos y reproducir escenarios con varios agentes sin perder coherencia dentro de la arquitectura distribuida adoptada.

2.10.4. Simulador Seleccionado

Atendiendo a las necesidades del proyecto, Gazebo ha sido el simulador seleccionado para el desarrollo y la validación del sistema de navegación propuesto. Esta elección responde a su amplia implantación en el ámbito de la robótica y, en particular, a su buena sintonía con infraestructuras basadas en ROS. Su motor físico, su capacidad de personalización y la integración

directa mediante paquetes como `gazebo_ros` y `gazebo_plugins` lo convierten en una herramienta especialmente adecuada para construir entornos de prueba coherentes con la arquitectura distribuida empleada en este trabajo.

Uno de los aspectos más relevantes de esta plataforma ha sido su integración fluida con MAVROS, lo que permite enlazar la simulación con un autopiloto virtual basado en PX4 o ArduPilot a través del protocolo MAVLink. De este modo, las consignas generadas por los nodos de control se envían al UAV virtual como si se tratara de un dron real, mientras que la telemetría producida por el autopiloto se redistribuye al resto de módulos ROS manteniendo la coherencia estructural del sistema. Esta continuidad entre percepción, control y simulación ha resultado especialmente valiosa para validar el comportamiento global de la arquitectura sin abandonar un entorno seguro y controlado.

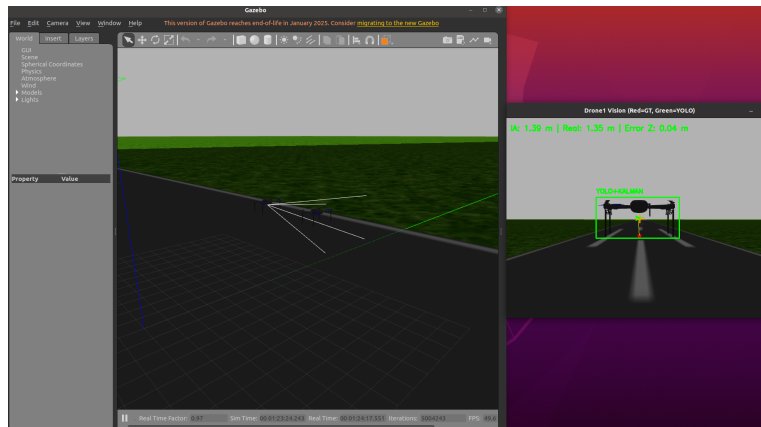


Figura 2.19. Uso de Gazebo con la cámara. (Imagen de Elaboración Propia).

En la práctica, Gazebo ha desempeñado un papel fundamental durante las fases de diseño, prueba y ajuste del sistema, ya que ha permitido analizar la respuesta del UAV ante cambios de trayectoria, maniobras de seguimiento y variaciones del entorno de forma repetible y sin riesgo para la plataforma física. Aun así, conviene recordar que la simulación no elimina por completo el denominado *sim2real gap*, es decir, las diferencias inevitables entre el entorno virtual y el comportamiento real del vehículo. Factores como las latencias, el ruido sensorial o ciertas dinámicas no modeladas obligan a interpretar la simulación como una etapa crítica de validación previa, pero no como un sustituto definitivo de las pruebas experimentales sobre hardware real.

Metodología de Trabajo

3.1. Configuración de ROS

En esta sección se describe el proceso seguido para configurar ROS Noetic [12] como base del entorno de desarrollo del proyecto. Para ello, se detalla la creación del espacio de trabajo, la instalación de los paquetes necesarios y el uso de los comandos básicos que permiten poner en marcha, supervisar y depurar los distintos nodos del sistema.

3.1.1. Creación de un Espacio de Trabajo

Para el desarrollo e integración de paquetes propios en ROS Noetic, resulta necesario disponer de un espacio de trabajo organizado, compatible con la estructura habitual de ROS y preparado para compilar los distintos módulos del proyecto. En este caso, se ha utilizado `catkin tools` en lugar del método tradicional basado en `catkin make`, ya que ofrece una gestión más flexible del proceso de compilación y facilita el control del entorno de desarrollo.

1. **Crear el directorio:** En primer lugar, se crea el directorio principal del espacio de trabajo, denominado `drone_ws`, junto con la carpeta `src`, destinada a almacenar el código fuente de los paquetes desarrollados o incorporados al sistema. Posteriormente, se inicializa el espacio de trabajo mediante las siguientes instrucciones:

```
mkdir -p ~/drone_ws/src
cd ~/drone_ws
catkin init
```

Con este procedimiento se establece la estructura básica del entorno de desarrollo y se deja preparada para su uso con `catkin tools`.

2. **Inicializar el espacio de trabajo:** Una vez creada e inicializada la estructura del workspace, se realiza una primera compilación con el objetivo de generar las carpetas `build` y `devel`, necesarias para construir los paquetes y configurar correctamente sus dependencias. Para ello, se emplea el siguiente comando:

```
catkin build
```

Durante esta etapa se generan los archivos asociados al proceso de compilación y se prepara el entorno que permitirá ejecutar los nodos, librerías y recursos vinculados al proyecto.

3. **Configurar el espacio de trabajo en el entorno:** Finalmente, para que ROS pueda localizar automáticamente los paquetes incluidos en `drone_ws` al abrir una nueva terminal, se añade el archivo de configuración del workspace al archivo `.bashrc` del usuario. Esta configuración se realiza mediante:

```
echo "source ~/drone_ws/devel/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

De este modo, cada nueva sesión de terminal cargará el entorno de ROS Noetic junto con los paquetes del espacio de trabajo `drone_ws`, evitando tener que ejecutar manualmente el comando `source` en cada ocasión.

3.1.2. Instalación de Paquetes

La correcta ejecución del sistema de navegación y seguimiento visual exige la integración de diversas bibliotecas especializadas que extienden las funcionalidades base de ROS Noetic. Para este proyecto, el entorno requiere herramientas que gestionen desde la comunicación con el hardware de vuelo hasta el procesamiento avanzado de imágenes y uso de matrices de transformación.

1. **Instalar MavROS y MavLink:** La instalación de *MavROS* y *MavLink* permite comunicar *ROS* con autopilotos como *ArduPilot* o *PX4*. Con ello se habilita el intercambio de telemetría, órdenes de navegación y datos de sensores dentro del sistema. El procedimiento se apoya en `wstool`, `rosdep` y `catkin build`, siguiendo la guía de *Intelligent Quads* [13]:

```
cd ~/drone_ws
wstool init ~/drone_ws/src

rosinstall_generator --upstream mavros | tee /tmp/mavros.rosinstall
rosinstall_generator mavlink | tee -a /tmp/mavros.rosinstall
wstool merge -t src /tmp/mavros.rosinstall
wstool update -t src
rosdep install --from-paths src --ignore-src --rosdistro 'echo $ROS_DISTRO' -y

catkin build
```

2. **Clonar el paquete *iq_sim* de *ROS*:** El paquete *iq_sim* [15], desarrollado por *Intelligent Quads*, proporciona modelos y escenarios de simulación para UAVs en *ROS* y *Gazebo*. En este proyecto se utiliza como base para generar un entorno de prueba controlado, incluyendo datos de cámara simulada mediante mensajes `sensor_msgs/Image`. Su descarga se realiza con los siguientes comandos:

```
cd ~/drone_ws/src
git clone https://github.com/Intelligent-Quads/iq_sim.git
```

3. **Instalar *ArduPilot* y *MAVProxy*:** *ArduPilot* se emplea como plataforma de control de vuelo para reproducir en simulación el comportamiento de un autopiloto real y representar esquemas de navegación convencionales. Por su parte, *MAVProxy* funciona como estación de control en terminal y permite interactuar con el UAV mediante *MAVLink*. La instalación se realiza con las siguientes instrucciones:

```
cd ~
sudo apt install git
git clone https://github.com/ArduPilot/ardupilot.git
cd ardupilot
Tools/environment_install/install-prereqs-ubuntu.sh -y

. ~/.profile

sudo pip install future pymavlink MAVProxy

export PATH=$PATH:$HOME/ardupilot/Tools/autotest
export PATH=/usr/lib/ccache:$PATH
. ~/.bashrc
```

4. **Instalar el plugin de *Gazebo* para *APM*:** El plugin *ardupilot_gazebo* permite enlazar *Gazebo* con *ArduPilot* para simular el vuelo del UAV con mayor realismo físico. Esta integración facilita el intercambio de datos entre el simulador y el autopiloto durante las pruebas. Su instalación se basa en la guía de *Intelligent Quads* [14]:

```
cd ~
git clone https://github.com/khancyr/ardupilot_gazebo.git
cd ardupilot_gazebo
mkdir -p build
cd build
cmake ..
```



```

make -j4
sudo make install

echo 'source "$(pkg-config --variable=prefix gazebo)/share/gazebo/setup.sh"' >> ~/.bashrc
echo 'export GAZEBO_MODEL_PATH=~/.ardupilot_gazebo/models' >> ~/.bashrc
. ~/.bashrc

```

5. **Instalación de herramientas para el procesamiento de imagen y visión:** Para tratar las imágenes obtenidas en la simulación se incorporan *cv_bridge*, *vision_opencv* y *OpenCV*. Estas herramientas permiten convertir mensajes de *ROS* a estructuras de imagen procesables y aplicar operaciones básicas de visión artificial.

```

sudo apt-get install ros-noetic-cv-bridge ros-noetic-vision-opencv libopencv-dev

```

6. **Instalación de librerías para cálculo matricial y álgebra lineal:** Para las operaciones matemáticas del sistema se instala *Eigen*, una biblioteca ligera y eficiente para trabajar con matrices, vectores y transformaciones geométricas.

```

sudo apt-get install libeigen-dev

```

Siguiendo estos pasos se obtiene un workspace provisto con todos los paquetes necesarios para el desarrollo satisfactorio del proyecto.

3.2. Configuración y Ejecución de la Simulación en Gazebo

En esta sección se presenta el procedimiento seguido para preparar y ejecutar la simulación de los UAVs mediante *Gazebo*. Este simulador físico permite representar entornos de prueba realistas e interactuar con nodos de *ROS*, ofreciendo un marco seguro y repetible para evaluar algoritmos de navegación, analizar la respuesta dinámica del vehículo y comprobar la robustez del sistema ante distintas condiciones de operación.

3.2.1. Modelo del Vehículo Aéreo y Configuración del Entorno

La simulación se construye a partir del paquete *iq_sim* [15], desarrollado por *Intelligent Quads*, ya que proporciona modelos de UAVs, sensores virtuales y mundos de prueba preparados para trabajar con *ROS* y *Gazebo*. El modelo empleado recoge los elementos necesarios para representar el comportamiento del dron dentro del simulador:

- **Estructura física:** define la geometría, masa, enlaces y articulaciones del vehículo, permitiendo reproducir su respuesta dinámica durante el vuelo.
- **Sensores integrados:** incorpora sensores simulados, como *IMU*, *GPS* y barómetro, cuyas medidas se publican en tópicos de *ROS* para ser utilizadas por los nodos de control.
- **Plugins de control:** emplea complementos de *Gazebo*, como `gazebo_ros_imu` o `gazebo_ros_control`, que facilitan la comunicación entre la física simulada y los procesos de control ejecutados en *ROS*.

Además del modelo del UAV, se utiliza un archivo de mundo `.world` para definir el escenario de pruebas. En este caso, se opta por un entorno aéreo abierto y sin obstáculos fijos, de modo que las trayectorias puedan desarrollarse con mayor libertad y sea posible observar el comportamiento del sistema en condiciones controladas.

3.2.2. Lanzamiento y Supervisión de la Simulación

La ejecución de la simulación se organiza mediante archivos `.launch`, encargados de cargar el modelo del dron, iniciar el entorno de *Gazebo* y activar los nodos necesarios para la comunicación con el sistema de control. Estos archivos se ajustan para incluir la carga automática del UAV, la inicialización de *MAVROS* como puente de comunicación y la conexión con *ArduPilot* a través de puertos UDP simulados.

Una vez iniciado el entorno, se activa el controlador correspondiente, ya sea el esquema convencional basado en *ArduPilot* o el control propuesto en el proyecto. Durante la ejecución se supervisa la respuesta del sistema mediante la visualización 3D en *Gazebo*, la consulta de tópicos con herramientas como `rqt_graph` y `rostopic echo`, y la inspección del estado de los nodos mediante `roslaunch list`. Esta supervisión permite comprobar que la simulación se ejecuta correctamente y que los módulos de percepción, comunicación y control intercambian información de forma adecuada.

```

1 <launch>
2   <!-- APM launch generico para multiples drones -->
3
4   <arg name="drone_id" default="1" />
5   <arg name="fcu_url" default="udp://127.0.0.1:14551@14555" /
6   >
7   <arg name="gcs_url" default="" />
8   <arg name="tgt_system" default="1" />
9   <arg name="tgt_component" default="1" />
10  <arg name="log_output" default="screen" />
11  <arg name="respawn_mavros" default="true" />
    <arg name="mavros_ns" default="/" />

```

```

12 <arg name="config_yaml"
13     default="$(env HOME)/drone_ws/mavros/launch/
14     apm_config_d$(arg drone_id).yaml" />
15
16 <include file="$(find iq_sim)/launch/mavros_node.launch">
17     <arg name="pluginlists_yaml"
18         value="$(find mavros)/launch/apm_pluginlists.yaml"
19     />
20     <arg name="config_yaml" value="$(arg config_yaml)" />
21     <arg name="mavros_ns" value="$(arg mavros_ns)" />
22     <arg name="fcu_url" value="$(arg fcu_url)" />
23     <arg name="gcs_url" value="$(arg gcs_url)" />
24     <arg name="respawn_mavros" value="$(arg respawn_mavros)"
25     " />
26     <arg name="tgt_system" value="$(arg tgt_system)" />
27     <arg name="tgt_component" value="$(arg tgt_component)"
28     />
29     <arg name="log_output" value="$(arg log_output)" />
30 </include>
31 </launch>

```

Listing 3.1: Archivo `apm.launch` para la conexión de *MAVROS*

3.3. Configuración UAVs en Simulación

En esta sección se aborda la configuración de los dos UAVs empleados en el entorno simulado. La separación entre ambos vehículos permite definir de forma clara sus parámetros de comunicación, identificadores, espacios de nombres y archivos de lanzamiento, evitando conflictos entre tópicos, nodos y conexiones con el autopiloto. Esta organización resulta especialmente importante cuando se trabaja con varios drones dentro de una misma instancia de *Gazebo*, ya que facilita el seguimiento individual de cada plataforma y prepara el sistema para ejecutar pruebas coordinadas de navegación y control.

3.3.1. Configuración del Dron 1

Para el primer UAV se tomó como base el modelo de dron incluido en *iq_sim* y se creó una variante propia denominada `drone1`, ubicada dentro de la ruta relativa `iq_sim/models`. Esta separación permite modificar el modelo sin alterar la plantilla original y facilita su identificación dentro del entorno de simulación. El modelo queda registrado mediante el archivo `model.config`, donde se declara el nombre `drone1` y se enlaza con el archivo principal `model.sdf`.

```

1 <model>
2   <name>drone1</name>

```

```

3   <version>1.0</version>
4   <sdf version="1.6">model.sdf</sdf>
5 </model>

```

Listing 3.2: Extracto de `model.config` para declarar el modelo `drone1`

En el archivo `model.sdf` se incorporó una cámara virtual fijada al cuerpo del vehículo, configurada para publicar imágenes y metadatos de calibración mediante el plugin `gazebo_ros_camera`.

```

1 <sensor name="cam" type="camera">
2   <pose>0 0 0.15 0 0 0</pose>
3   <camera>
4     <horizontal_fov>1.2</horizontal_fov>
5     <image>
6       <width>640</width>
7       <height>480</height>
8     </image>
9     <clip>
10      <near>0.1</near>
11      <far>1000</far>
12    </clip>
13  </camera>
14  <always_on>1</always_on>
15  <update_rate>30</update_rate>
16  <visualize>true</visualize>
17  <plugin name="camera_controller" filename="
18    libgazebo_ros_camera.so">
19    <always_on>true</always_on>
20    <update_rate>30</update_rate>
21    <cameraName>webcam</cameraName>
22    <imageTopicName>image_raw</imageTopicName>
23    <cameraInfoTopicName>camera_info</cameraInfoTopicName>
24    <frameName>camera_link</frameName>
25  </plugin>
26 </sensor>
27 </link>
28 <joint name="camera_mount" type="fixed">
29   <child>cam_link</child>
30   <parent>iris::base_link</parent>
31   <axis>
32     <xyz>0 0 1</xyz>
33     <limit>
34       <upper>0</upper>
35       <lower>0</lower>
36     </limit>
37   </axis>
38 </joint>

```

Listing 3.3: Extracto de código en `model.sdf`, sensor de cámara incorporado para `drone1`

Además del modelo físico, se generó una carpeta `config` asociada a `drone1`. En ella se incluyó un archivo `.yaml` con la información de la cámara, necesario para mantener organizada su configuración dentro de *ROS*. También se añadió el archivo `drone1_params.parm`, donde se define el identificador del vehículo para que *ArduPilot* pueda reconocerlo de forma independiente dentro de la simulación.

```
1 SYSID_THISMAV 1
```

Listing 3.4: Identificador asignado al primer UAV en `drone1_params.parm`

Por último, se creó el archivo `apm_config_d1.yaml` dentro de la ruta relativa `mavros/launch`. Este archivo es leído por `apm.launch` mediante el argumento `config.yaml` y permite lanzar el primer dron con marcos de referencia propios. En particular, se asigna el cuerpo del UAV al marco `drone1/base_link`, evitando solapamientos con otros vehículos simulados.

```
1 # global_position
2 global_position:
3   frame_id: "map"
4   child_frame_id: "drone1/base_link"
5   rot_covariance: 99999.0
6   gps_uere: 1.0
7   use_relative_alt: true
8   tf:
9     send: false
10    frame_id: "map"
11    global_frame_id: "earth"
12    child_frame_id: "drone1/base_link"
13
14 # imu_pub
15 imu:
16   frame_id: "drone1/base_link"
17   linear_acceleration_stdev: 0.0003
18   angular_velocity_stdev: 0.0003490659
19   orientation_stdev: 1.0
20   magnetic_stdev: 0.0
21
22 # local_position
23 local_position:
24   frame_id: "map"
25   tf:
26     send: true
27     frame_id: "map"
28     child_frame_id: "drone1/base_link"
29     send_fcu: false
```

Listing 3.5: Fragmento principal de `apm_config_d1.yaml` para `drone1`

3.3.2. Configuración del Dron 2

Para el segundo UAV se creó una nueva carpeta denominada **drone2**, tomando como referencia la estructura ya definida para **drone1**. Esta copia permitió mantener una organización homogénea entre ambos modelos, pero eliminando la cámara del segundo vehículo, ya que en esta configuración no era necesario disponer de un segundo flujo de imagen. De este modo, **drone2** conserva la estructura básica del UAV y queda preparado para ejecutarse de forma simultánea dentro de *Gazebo* sin interferir con el primer dron.

Al igual que en el caso anterior, el modelo se registra mediante el archivo **model.config**. En él se define el nombre del modelo como **drone2** y se indica que la descripción física del vehículo debe leerse desde el archivo **model.sdf**. Este archivo contiene la estructura del dron, sus enlaces, propiedades físicas y elementos necesarios para que *Gazebo* pueda cargarlo correctamente en la simulación.

```
1 <model>
2   <name>drone2</name>
3   <version>1.0</version>
4   <sdf version="1.6">model.sdf</sdf>
5 </model>
```

Listing 3.6: Extracto de **model.config** para declarar el modelo **drone2**

Asimismo, se creó un archivo de parámetros propio, denominado **drone2_params.parm**, para asignar un identificador independiente al segundo UAV. Este valor permite que *ArduPilot* distinga ambos vehículos durante la ejecución y evita conflictos en la comunicación con *MAVROS*.

```
1 SYSID_THISMAV 2
```

Listing 3.7: Identificador asignado al segundo UAV en **drone2_params.parm**

Por último, se creó el archivo **apm_config_d2.yaml** dentro de la ruta relativa **mavros/launch**. Este archivo cumple la misma función que el definido para el primer UAV, pero empleando los marcos de referencia propios de **drone2**. En concreto, se asigna el cuerpo del vehículo al marco **drone2/base_link**, permitiendo que *MAVROS* publique la posición global, la información inercial y la posición local sin solaparse con los datos del primer dron.

```
1 # global_position
2 global_position:
3   frame_id: "map" # origin frame
4   child_frame_id: "drone2/base_link" # body-fixed frame
5   rot_covariance: 99999.0 # covariance for attitude?
6   gps_uere: 1.0 # User Equivalent Range Error (UERE
   ) of GPS sensor (m)
```

```

7   use_relative_alt: true      # use relative altitude for local
    coordinates
8   tf:
9     send: false              # send TF?
10    frame_id: "map"          # TF frame_id
11    global_frame_id: "earth" # TF earth frame_id
12    child_frame_id: "drone2/base_link" # TF child_frame_id
13
14 # imu_pub
15 imu:
16   frame_id: "drone2/base_link"
17   # need find actual values
18   linear_acceleration_stdev: 0.0003
19   angular_velocity_stdev: 0.0003490659 // 0.02 degrees
20   orientation_stdev: 1.0
21   magnetic_stdev: 0.0
22
23 # local_position
24 local_position:
25   frame_id: "map"
26   tf:
27     send: true
28     frame_id: "map"
29     child_frame_id: "drone2/base_link"
30     send_fcu: false

```

Listing 3.8: Fragmento principal de `apm_config_d2.yaml` para `drone2`

Tras definir los modelos y sus configuraciones individuales, ambos UAVs fueron añadidos al archivo de mundo `runaway.world`. En este archivo se indica a *Gazebo* qué modelos deben cargarse en la escena y la posición inicial de cada uno. De esta forma, `drone1` se sitúa en el origen del entorno, mientras que `drone2` se inicializa desplazado 10 metros en el eje x , evitando que ambos vehículos aparezcan superpuestos al comenzar la simulación.

```

1 <model name="drone1">
2   <pose>0 0 0 0 0 0</pose>
3   <include>
4     <uri>model://drone1</uri>
5   </include>
6 </model>
7
8 <model name="drone2">
9   <pose>10 0 0 0 0 0</pose>
10  <include>
11    <uri>model://drone2</uri>
12  </include>
13 </model>

```

Listing 3.9: Inclusión de `drone1` y `drone2` en `runaway.world`

3.3.3. Ejecución Básica de la Simulación

Una vez definidos los modelos `drone1` y `drone2`, sus identificadores de *ArduPilot*, los archivos de configuración de *MAVROS* y su inclusión en el mundo de *Gazebo*, la simulación básica se ejecuta mediante varias terminales simultáneas. Para organizar este proceso se empleó *Terminator*, una herramienta que permite dividir una misma ventana en varios paneles de terminal. Esta característica resulta especialmente útil en entornos ROS, ya que facilita observar al mismo tiempo la salida de *Gazebo*, las dos instancias SITL de *ArduPilot* y los dos nodos de *MAVROS*, evitando cambiar continuamente entre ventanas independientes.

The image shows a Terminator window with three terminal panes. The top pane shows the execution of `roslaunch iq_sim runway.launch`, which starts the Gazebo world. The middle pane shows the execution of `sim_vehicle.py -v ArduCopter -f gazebo-iris --console -IO \ --out=tcpin:0.0.0.0:8100 \ --add-param-file=$HOME/drone_ws/src/iq_sim/models/drone1/drone1_params.parm`, which starts the first SITL instance. The bottom pane shows the execution of `sim_vehicle.py -v ArduCopter -f gazebo-iris --console -IO \ --out=tcpin:0.0.0.0:8100 \ --add-param-file=$HOME/drone_ws/src/iq_sim/models/drone2/drone2_params.parm`, which starts the second SITL instance. The output of the bottom pane shows the initialization of the SITL instance, including the loading of the `ArduCopter` model and the configuration of the `gazebo-iris` environment.

Figura 3.1. Panel básico de ejecución con varias terminales organizadas en *Terminator*. Imagen de elaboración propia.

En la Figura 3.1 se muestra la disposición básica utilizada para lanzar el sistema. Cada panel corresponde a una tarea concreta y se mantiene visible durante toda la prueba, lo que permite detectar rápidamente errores de conexión, problemas en el arranque del autopiloto o fallos en la publicación de mensajes. La ejecución se organiza de la siguiente forma:

```
# Terminal 1: lanzamiento del mundo de Gazebo
roslaunch iq_sim runway.launch
```

```
# Terminal 2: instancia SITL del primer UAV
sim_vehicle.py -v ArduCopter -f gazebo-iris --console -IO \
--out=tcpin:0.0.0.0:8100 \
--add-param-file=$HOME/drone_ws/src/iq_sim/models/drone1/drone1_params.parm
```

```
# Terminal 3: instancia SITL del segundo UAV
```



```
sim_vehicle.py -v ArduCopter -f gazebo-iris --console -I1 \
  --out=tcpin:0.0.0.0:8200 \
  --add-param-file=$HOME/drone_ws/src/iq_sim/models/drone2/drone2_params.parm
```

```
# Terminal 4: conexion MAVROS del primer UAV
roslaunch iq_sim apm.launch drone_id:=1 \
  fcu_url:=udp://127.0.0.1:14551@14555 \
  mavros_ns:=/drone1 tgt_system:=1
```

```
# Terminal 5: conexion MAVROS del segundo UAV
roslaunch iq_sim apm.launch drone_id:=2 \
  fcu_url:=udp://127.0.0.1:14561@14565 \
  mavros_ns:=/drone2 tgt_system:=2
```

En la primera terminal se lanza el entorno de *Gazebo* mediante `runway.launch`, que carga el mundo preparado previamente y permite visualizar los modelos incorporados en `runaway.world`. Las terminales segunda y tercera arrancan dos instancias independientes de *ArduPilot* en modo SITL. La opción `-IO` se asocia al primer UAV y la opción `-I1` al segundo, mientras que los archivos `drone1_params.parm` y `drone2_params.parm` asignan los identificadores `SYSID_THISMAV 1` y `SYSID_THISMAV 2`, respectivamente. De este modo, cada autopiloto virtual queda vinculado al dron configurado en las subsecciones anteriores.

Las terminales cuarta y quinta ejecutan el archivo `apm.launch` para crear los dos puentes *MAVROS*. En cada caso se indica el identificador del dron mediante `drone_id`, el puerto UDP correspondiente mediante `fcu_url`, el espacio de nombres `mavros_ns` y el sistema objetivo mediante `tgt_system`. Esta separación permite que los tópicos, servicios y marcos de referencia de `drone1` y `drone2` se mantengan diferenciados dentro de ROS, coherentemente con los archivos `apm_config_d1.yaml` y `apm_config_d2.yaml` descritos anteriormente.

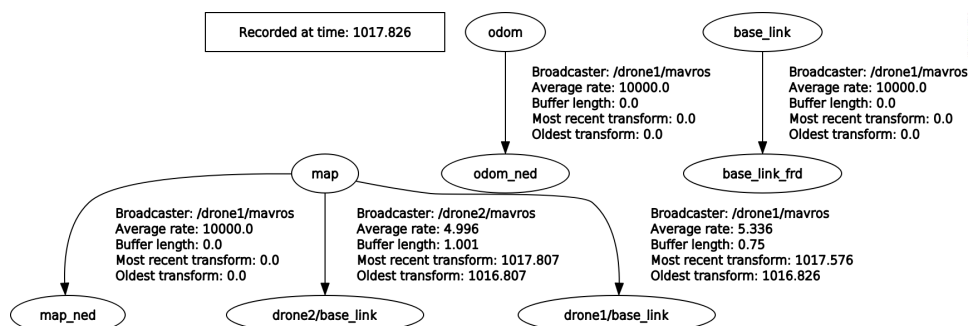


Figura 3.2. Marcos de referencia publicados durante la ejecución inicial con dos UAVs. Imagen de elaboración propia.

La Figura 3.2 muestra los marcos de referencia publicados tras ejecutar los comandos anteriores. Esta comprobación permite verificar que cada UAV dispone de su propio *frame* asociado, como `drone1/base_link` y `drone2/base_link`, enlazado con el marco global `map`. De esta forma se confirma que la configuración definida en `apm_config_d1.yaml` y `apm_config_d2.yaml` separa correctamente las transformaciones de ambos vehículos y evita ambigüedades al incorporar posteriormente los nodos de percepción y control propuestos.

3.4. Configuración del Control Propuesto y Ejecución en Gazebo

Implementacion

4.1. Solucion Software

4.1.1. Preparación del Espacio de Trabajo

4.1.2. Solución para Simulación

4.1.3. Desarrollo de Scripts y Gráficas

4.2. Arquitectura del Sistema

4.3. Modulo de percepcion

4.3.1. Eleccion de modelo de Yolo

4.3.2. USo Yolo

4.3.3. Uso de Filtro de Kalman y EMA

4.4. Geometría Proyectiva

El modelo de cámara estenopeica (*pinhole camera model*) idealiza la formación de la imagen suponiendo que todos los rayos de luz atraviesan un único punto, denominado centro óptico o centro de proyección, antes de alcanzar el plano imagen. Esta simplificación elimina el comportamiento complejo de las lentes reales y permite describir la proyección mediante relaciones geométricas limpias y fácilmente interpretables [65, 64].

En una cámara física, la imagen se forma invertida sobre el sensor. No obstante, en visión por computador suele emplearse un plano imagen virtual situado delante del centro óptico para evitar inversiones de signo y hacer más intuitiva la interpretación matemática del modelo. Esta convención no altera la geometría del problema, pero simplifica el análisis de la proyección y la definición de los errores visuales [65, 63].

Si un punto del entorno se expresa en el sistema de referencia de la cámara como $P_c = (X_c, Y_c, Z_c)$, su proyección ideal sobre el plano imagen métrico viene dada por:

$$x = f \frac{X_c}{Z_c}, \quad y = f \frac{Y_c}{Z_c} \quad (4.1)$$

En esta expresión, X_c e Y_c representan la posición lateral y vertical del punto respecto al eje óptico, Z_c es su profundidad o distancia a lo largo del eje de visión, f es la distancia focal y (x, y) son las coordenadas proyectadas sobre el plano imagen ideal. La ecuación pone de manifiesto el comportamiento esencial de la perspectiva: cuanto mayor es Z_c , menor es la proyección del objeto en la imagen, haciendo que los objetos lejanos aparecen más pequeños y los cercanos ocupan más píxeles [64, 63].

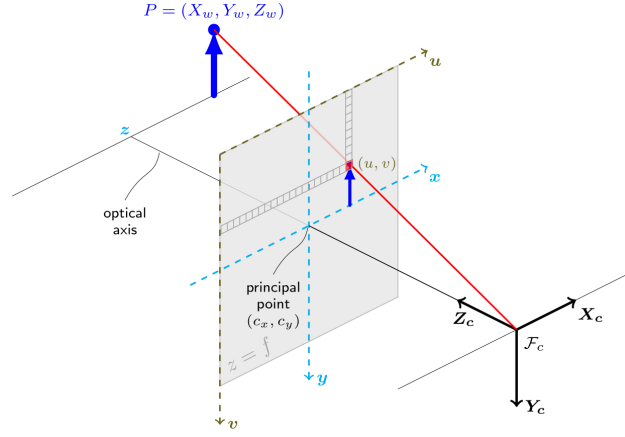


Figura 4.1. Esquema simplificado del modelo de cámara estenopeica. Fuente: [59].

4.4.1. Fundamentos de la Geometría Proyectiva

Para trabajar de forma práctica con imágenes digitales, la proyección anterior se expresa normalmente en coordenadas normalizadas y, posteriormente, en coordenadas de píxel. Las coordenadas normalizadas vienen dadas por:

$$x_n = \frac{X_c}{Z_c}, \quad y_n = \frac{Y_c}{Z_c} \quad (4.2)$$

Estas variables recogen la parte puramente proyectiva del problema: dependen de la dirección del punto respecto a la cámara, pero no del tamaño físico del sensor. A partir de ellas, el paso a píxeles se modela como:

$$u = f_x x_n + c_x, \quad v = f_y y_n + c_y \quad (4.3)$$

Aquí, f_x y f_y son las distancias focales expresadas en píxeles en las direcciones horizontal y vertical, mientras que c_x y c_y definen el punto principal, es decir, la proyección del eje óptico sobre la imagen. En una cámara ideal, este punto suele estar cerca del centro del sensor, aunque en la práctica puede aparecer ligeramente desplazado por el montaje o la calibración [63, 59].

La formulación completa en coordenadas homogéneas se resume mediante la ecuación:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \cdot [R|t] \cdot \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (4.4)$$

En esta relación, (X_w, Y_w, Z_w) son las coordenadas del punto en el sistema de referencia del mundo, $[R|t]$ describe la transformación extrínseca que lleva ese punto al sistema de referencia de la cámara, K es la matriz de parámetros intrínsecos y s es un factor de escala propio de las coordenadas homogéneas. La matriz intrínseca se suele escribir como:

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (4.5)$$

Cada término tiene una interpretación física clara: f_x y f_y convierten coordenadas geométricas en coordenadas de píxel, c_x y c_y fijan el punto principal y γ representa el sesgo o *skew* entre ejes de píxel, normalmente nulo o despreciable en cámaras modernas. Esta descomposición es importante porque separa dos preguntas distintas: dónde está la cámara respecto al mundo (parámetros extrínsecos) y cómo proyecta la cámara lo que ve (parámetros intrínsecos) [63, 65].

4.4.2. Parámetros intrínsecos, FOV y distorsión

Otro concepto fundamental es el campo de visión o *Field of View* (FOV), que puede entenderse como la amplitud angular de la escena que la cámara es capaz de cubrir. Si d_x y d_y representan el tamaño del sensor en horizontal y vertical, entonces el FOV puede aproximarse mediante:

$$FOV_x = 2 \arctan \left(\frac{d_x}{2f} \right), \quad FOV_y = 2 \arctan \left(\frac{d_y}{2f} \right) \quad (4.6)$$

Estas expresiones pueden interpretarse de forma sencilla. Para un mismo sensor, una distancia focal pequeña genera un FOV amplio: la cámara ve una porción mayor de la escena, pero cada objeto ocupa menos píxeles. En cambio, una distancia focal grande produce un FOV más estrecho: la cámara cubre menos escena, aunque el objetivo aparece con mayor tamaño aparente en la imagen. En tareas de seguimiento, este comportamiento introduce un compromiso claro entre cobertura y detalle: un FOV amplio ayuda a no perder el blanco durante maniobras bruscas, mientras que un FOV estrecho permite medir con más precisión sus desplazamientos cuando se encuentra lejos [64, 63].

Hasta aquí se ha descrito una cámara ideal. Sin embargo, una cámara real no proyecta la escena de manera perfectamente geométrica, porque la lente introduce pequeñas deformaciones. Las más relevantes en visión por computador son la distorsión radial y la distorsión tangencial. La distorsión radial depende sobre todo de la distancia al centro de la imagen: cuanto más se aleja un punto de esa zona central, mayor puede ser su desplazamiento aparente, lo que hace que las rectas se curven cerca de los bordes. La distorsión tangencial, por su parte, aparece cuando la lente y el sensor no están perfectamente alineados y produce desplazamientos asimétricos de los puntos observados [59, 65].

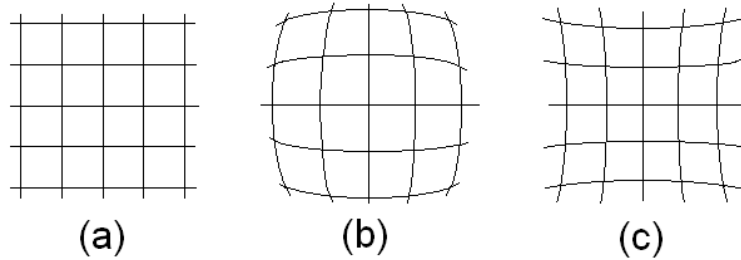


Figura 4.2. Imágenes de una pantalla de objeto rectangular mostradas con (a) ninguna distorsión; (b) distorsión radial; (c) distorsión tangencial., [66].

En coordenadas normalizadas, una forma habitual de modelar esta distorsión es la siguiente:

$$r^2 = x_n^2 + y_n^2 \quad (4.7)$$

$$x_d = x_n \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right) + 2p_1 x_n y_n + p_2 \left(r^2 + 2x_n^2 \right) \quad (4.8)$$

$$y_d = y_n \left(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \right) + p_1 \left(r^2 + 2y_n^2 \right) + 2p_2 x_n y_n \quad (4.9)$$

En estas ecuaciones, r mide la distancia del punto al centro de la imagen normalizada; los coeficientes k_1 , k_2 y k_3 describen la parte radial de la deformación, mientras que p_1 y p_2 describen la parte tangencial. Los puntos (x_d, y_d) representan, por tanto, la posición finalmente observada tras el efecto de la lente. La consecuencia práctica es directa: una misma distancia física en la escena no siempre se traduce en la misma distancia en píxeles si la medida se toma en regiones distintas de la imagen [59].

Desde el punto de vista del control, la distorsión introduce una dispersión en la medida visual: un mismo movimiento real puede producir errores distintos según la zona de la imagen en la que aparezca el objetivo. Por ello, para alimentar al controlador PID conviene trabajar con puntos sin distorsión o con coordenadas normalizadas obtenidas tras calibrar la cámara [59, 63].

En este trabajo, corregir esos puntos permite estimar mejor las distancias aparentes y generar una señal de error más estable para el lazo de control. Esta idea enlaza con la sección siguiente, dedicada a cómo el controlador PID transforma ese error visual en comandos de movimiento del UAV cazador.

4.5. Filtro de Kalman

El filtro de Kalman es un método de estimación recursiva empleado para obtener una aproximación del estado de un sistema dinámico cuando las medidas disponibles contienen ruido. Su utilidad, en este trabajo, reside en que permite combinar la información procedente del modelo con la medida visual del objetivo, generando una estimación más estable y más útil para el controlador PID [52, 53, 54].



Figura 4.3. Ejemplo uso de Filtro de Kalman. (Elaboración propia)

En primer lugar, conviene aclarar qué se entiende por **vector de estado**. El vector de estado es el conjunto de variables que describen la situación del sistema en un instante dado. En un problema de seguimiento visual, este vector puede incluir, por ejemplo, la posición del objetivo en la imagen y su velocidad aparente. Una forma podría ser:

$$\mathbf{x}_k = \begin{bmatrix} x_k & y_k & \dot{x}_k & \dot{y}_k \end{bmatrix}^T \quad (4.10)$$

En este caso, x_k e y_k representan la posición del objetivo en el plano imagen, mientras que $\dot{x}_k$ y $\dot{y}_k$ representan su velocidad aparente. A partir de este vector, el modelo describe tanto la evolución temporal del sistema como la relación entre el estado interno y la medida observada [53, 54].

El sistema dinámico lineal discreto puede escribirse mediante las ecuaciones:

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (4.11)$$

$$\mathbf{z}_k = \mathbf{H}_k \mathbf{x}_k + \mathbf{v}_k \quad (4.12)$$

En estas expresiones, la matriz $\mathbf{F}_k$ corresponde a la **matriz de transición de estado**, encargada de modelar la evolución del sistema entre dos instantes consecutivos. Por su parte, la matriz $\mathbf{H}_k$ corresponde a la **matriz de observación**, cuya función es relacionar el vector de estado con la magnitud efectivamente medida por el sensor. En términos conceptuales, $\mathbf{F}_k$ describe la dinámica del sistema, mientras que $\mathbf{H}_k$ determina qué componentes de dicho estado son accesibles a través de la cámara o del sensor empleado [53, 54].

Por ejemplo, si se adopta un modelo bidimensional de velocidad constante con periodo de muestreo Δt , y la cámara mide únicamente la posición del objetivo, unas elecciones habituales son:

$$\mathbf{F}_k = \begin{bmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{H}_k = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \quad (4.13)$$

Otro aspecto fundamental es el papel de las matrices de covarianza $\mathbf{Q}_k$ y $\mathbf{R}_k$. La matriz $\mathbf{Q}_k$ representa la incertidumbre asociada al modelo dinámico, es decir, cuánto puede desviarse la evolución real del sistema respecto a la

predicción teórica. La matriz $\mathbf{R}_k$, en cambio, representa la incertidumbre de la medida, es decir, el nivel de ruido presente en la observación del sensor. En términos prácticos, $\mathbf{Q}_k$ expresa cuánto se confía en el modelo y $\mathbf{R}_k$ cuánto se confía en la medida [53, 54].

En el mismo ejemplo anterior, una forma sencilla de representar ambas matrices sería:

$$\mathbf{Q}_k = \begin{bmatrix} \sigma_x^2 & 0 & 0 & 0 \\ 0 & \sigma_y^2 & 0 & 0 \\ 0 & 0 & \sigma_{\dot{x}}^2 & 0 \\ 0 & 0 & 0 & \sigma_{\dot{y}}^2 \end{bmatrix}, \quad \mathbf{R}_k = \begin{bmatrix} \sigma_{mx}^2 & 0 \\ 0 & \sigma_{my}^2 \end{bmatrix} \quad (4.14)$$

Aquí, los términos de $\mathbf{Q}_k$ recogen la incertidumbre asociada a la posición y a la velocidad del modelo, mientras que los términos de $\mathbf{R}_k$ representan el ruido de la medida en los ejes horizontal y vertical de la imagen.

Antes de iniciar el proceso iterativo del filtro, también es necesario fijar unas **condiciones iniciales**. En concreto, se requiere una estimación inicial del estado, denotada por $\hat{\mathbf{x}}_{0|0}$, y una estimación inicial de la incertidumbre asociada, representada por la matriz $\mathbf{P}_{0|0}$. Si se dispone de poca información inicial, la covarianza inicial suele elegirse grande; si, por el contrario, el estado inicial es relativamente conocido, esa covarianza puede fijarse con valores menores. Estas condiciones iniciales determinan el punto de partida a partir del cual el filtro comienza a refinar sus estimaciones.

Una vez definidos el estado, el modelo y las incertidumbres, el filtro opera en dos fases: **predicción** y **actualización**. En la fase de predicción, se calcula cuál debería ser el estado del sistema en el instante siguiente y cuál es la incertidumbre asociada a esa predicción:

$$\hat{\mathbf{x}}_{k|k-1} = \mathbf{F}_k \hat{\mathbf{x}}_{k-1|k-1} + \mathbf{B}_k \mathbf{u}_k \quad (4.15)$$

$$\mathbf{P}_{k|k-1} = \mathbf{F}_k \mathbf{P}_{k-1|k-1} \mathbf{F}_k^T + \mathbf{Q}_k \quad (4.16)$$

La primera ecuación proporciona la predicción del estado y la segunda actualiza la incertidumbre de dicha predicción. En esta etapa, el filtro todavía no ha incorporado la medida real del instante k .

Posteriormente, en la fase de actualización, la predicción se compara con la medida observada. La diferencia entre ambas se denomina **innovación** y se expresa como:

$$\tilde{\mathbf{y}}_k = \mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_{k|k-1} \quad (4.17)$$

A partir de esta diferencia, se calcula la ganancia de Kalman:

$$\mathbf{K}_k = \mathbf{P}_{k|k-1} \mathbf{H}_k^T \left(\mathbf{H}_k \mathbf{P}_{k|k-1} \mathbf{H}_k^T + \mathbf{R}_k \right)^{-1} \quad (4.18)$$

Esta ganancia determina cuánto peso debe darse a la medida y cuánto a la predicción. Con ella, se actualiza el estado estimado y su covarianza:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \mathbf{K}_k \tilde{\mathbf{y}}_k \quad (4.19)$$

$$\mathbf{P}_{k|k} = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \mathbf{P}_{k|k-1} \quad (4.20)$$

La lógica del método es sencilla: si la medida es poco fiable, el filtro confía más en la predicción del modelo; si la observación es consistente, la corrección adquiere mayor peso. Como resultado, se obtiene una estimación más suave y más coherente del movimiento del objetivo, lo que resulta especialmente útil cuando existen maniobras rápidas, pequeñas oclusiones o fluctuaciones en la detección visual [53, 54].

4.6. Compensacion de Actitud y proyecccion

4.7. Control de Seguimiento

4.7.1. Control PID

4.7.2. Control de distancia

4.8. Configuración del Control Propuesto y Ejecución en Gazebo

4.9. Metricas

Pruebas y Validaciones

- 5.1. Planificación de las Pruebas
- 5.2. Métricas de Evaluación
- 5.3. Desarrollo de los Experimentos
 - 5.3.1. Seguimiento del Objetivo
- 5.4. Análisis de Resultados
- 5.5. Experimentos

Conclusiones

- 6.1. Revisión del cumplimiento de objetivos
- 6.2. Conclusiones finales
- 6.3. Futuros Trabajos

Bibliografía

- [1] Instituto Nacional de Técnica Aeroespacial (INTA).
Logotipo del INTA.
URL: <https://www.inta.es/INTA/es/index.html>.
- [2] Robotics Operating Sy.
Logotipo de ROS.
URL: <https://www.skyfilabs.com/blog/10-simple-ros-projects-for-beginners>.
- [3] Jobted.
Salario promedio para ingeniero de software en España.
2025.
URL:
<https://www.jobted.es/salario/ingeniero-software>.
- [4] Dron de ala fija.
Foto de dron ala fija.
URL: <https://umilesgroup.com/dron-ala-fija/>.
- [5] Dron VTOL.
Foto de dron VTOL.
URL: <https://www.foxtechuav.com/es/inspection-vtol-drone-long-endurance-2500mm-wingspan-ayk-250.html>.
- [6] Dron multirotores.
Foto de dron multirotores.
URL: <https://www.unmannedsystemstechnology.com/es/expo/multirotor-drones/>.
- [7] clasificación drones 1.
clasificación drones 1.
URL: <https://umilesgroup.com/tipos-de-drones/>.
- [8] clasificación drones 2.
clasificación drones 2.
URL: <https://www.embention.com/es/noticias/tipos-de-drones-y-sus-principales-ventajas/>.

BIBLIOGRAFÍA

- [9] S. Jana, L. A. Tony, A. A. Bhise, V. VP y D. Ghose.
Interception of an aerial manoeuvring target using monocular vision.
Robotica, vol. 40, n.º 12, diciembre de 2022, pp. 4535–4554.
Publicado en línea el 03 de agosto de 2022. DOI:
<https://doi.org/10.1017/S0263574722001096>. URL:
<https://www.cambridge.org/core/journals/robotica/article/interception-of-an-aerial-manoeuving-target-using-monocular-vision/45EC80975FCD5727049599E55622B446>.
- [10] Juan Gabriel Gomila.
Problema del desvanecimiento del gradiente en IA.
Frogames Formación, 04/09/2020. URL:
<https://cursos.frogamesformacion.com/pages/blog/problema-del-desvanecimiento-del-gradiente-en-ia>.
- [11] Adrián Hernández.
Problema del desvanecimiento del gradiente (vanishing gradient problem).
Meta-learning, 6 de mayo de 2018. URL:
<https://stalearning.wordpress.com/2018/05/06/problema-de-desvanecimiento-del-gradiente-vanishing-gradient-problem/>.
- [12] .
Enlace descarga Ros Noetic.
Enlace descarga Ros Noetic:
<https://wiki.ros.org/noetic/Installation/Ubuntu>.
- [13] Intelligent Quads.
Installing ROS on Ubuntu 20.04.
GitHub, guía de instalación. URL:
https://github.com/Intelligent-Quads/iq_tutorials/blob/master/docs/installing_ros_20_04.md.
- [14] Intelligent Quads.
Installing Gazebo ArduPilot Plugin.
GitHub, guía de instalación. URL:
https://github.com/Intelligent-Quads/iq_tutorials/blob/master/docs/installing_gazebo_ardupilotplugin.md.
- [15] Intelligent Quads.
github de iqsim:
https://github.com/Intelligent-Quads/iq_sim.

BIBLIOGRAFÍA

- [16] I. Goodfellow, Y. Bengio y A. Courville.
Deep Learning.
MIT Press, 2016. URL:
<https://www.deeplearningbook.org/>.
- [17] Nanobaly.
Introducción a la detección de objetos en visión artificial [2025].
Innovatiana, publicado el 2024-01-26. URL: <https://www.innovatiana.com/es/post/object-detection-our-guide>.
- [18] Imagen Dron monitoreando cultivos.
Imagen Dron monitoreando cultivos.
URL: <https://uss.com.ar/corporativo/aplicaciones-de-drones-para-monitoreo-y-vigilancia-de-grandes-areas/>.
- [19] Web sobre Dron en la agricultura.
Web sobre Dron en la agricultura.
URL: <https://www.muginuav.com/es/drones-in-agriculture-overview/>.
- [20] Imagen Dron Rescate Maritimo.
Imagen Dron Rescate Maritimo.
URL: <https://www.drone-school.es/contratar-pilotos-y-servicios-con-drones/servicios-con-drone-para-salvamento-y-busqueda/>.
- [21] Web el uso del dron en emergencias y busqueda .
Web el uso del dron en emergencias y busqueda .
URL: <https://aerocamaras.es/servicios-drones-profesionales/emergencias-y-rescates-con-drones/>.
- [22] Imagen Dron transportando paquetes.
Imagen Dron transportando paquetes.
URL: <https://easy-trans.es/el-actual-uso-de-los-drones-en-el-transporte/>.
- [23] Web el uso del dron en logistica , trasporte.
Web el uso del dron en logistica , trasporte .
URL: <https://idc.apddrones.com/transporte/uso-de-drones-en-transporte-y-logistica/>.
- [24] Imagen Dron uso en defensa.
Imagen Dron uso en defensa .
URL: <https://geopol21.com/como-ha-evolucionado-la-guerra-con-drones-autonomos/>.

BIBLIOGRAFÍA

- [25] Web el uso del dron en defensa.
Web el uso del dron en defensa.
URL: <https://sentrycs.com/es/the-counter-drone-blog/empowering-special-forces-with-counter-drone-solutions/>.
- [26] K. Chávez y O. Swed.
Emulating underdogs: Tactical drones in the Russia-Ukraine war.
2024. URL: <https://www.tandfonline.com/doi/full/10.1080/13523260.2023.2257964>.
- [27] Wikipedia.
Visual servoing.
URL: https://en.wikipedia.org/wiki/Visual_servoing.
- [28] F. Chaumette y S. Hutchinson.
Visual servo control, part I: Basic approaches.
IEEE Robotics & Automation Magazine, vol. 13, n.º 4, 2006.
URL: <https://inria.hal.science/inria-00350283>.
- [29] K. Yang, C. Bai, Z. She y Q. Quan.
High-speed interception multicopter control by image-based visual servoing.
IEEE Trans. Control Syst. Technol., 2024. URL: <http://arxiv.org/abs/2404.08296>.
- [30] K. Yang y Q. Quan.
An autonomous intercept drone system with image-based visual servoing via perspective dynamic filtering.
2020 IEEE International Conference on Robotics and Automation (ICRA), Paris, France, 2020, pp. 3944–3950. DOI: <https://doi.org/10.1109/ICRA40945.2020.9197539>. URL: <https://ieeexplore.ieee.org/document/9197539>.
- [31] H. Yan, K. Yang, Y. Cheng, Z. Wang y D. Li.
Precise interception flight targets by image-based visual servoing of multicopter.
Sep. 2024. URL: <http://arxiv.org/abs/2409.17497>.
- [32] P. M. Wyder et al.
Autonomous drone hunter operating by deep learning and all-onboard computations in GPS-denied environments.
PLOS ONE, vol. 14, 2019. URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0225092>.

BIBLIOGRAFÍA

- [33] Z. W. Lee, W. H. Chin y H. W. Ho.
Air-to-air micro air vehicle interceptor with an embedded mechanism and deep learning.
Aerospace Science and Technology, vol. 135, 2023. URL:
<https://www.sciencedirect.com/science/article/pii/S1270963823000895>.
- [34] A. Rodriguez-Ramos, A. Alvarez-Fernandez, H. Bavle, J. Rodriguez-Vazquez, L. Lu, M. Fernandez-Cortizas, R. A. Suarez Fernandez, A. Rodelgo, C. Santos, M. Molina, L. Merino, F. Caballero y P. Campoy.
Autonomous Aerial Robot for High-Speed Search and Intercept Applications.
arXiv preprint arXiv:2112.05465, 2021. URL:
<https://arxiv.org/abs/2112.05465>.
- [35] C.A.G. van der Aa.
Design of an Image-Based Visual Servoing System for Autonomous Quadcopter Interception.
Master Thesis, Delft University of Technology, 2024. URL:
<https://resolver.tudelft.nl/uuid:4d8f5709-c20e-4ed2-9f62-26ce3ecde223>.
- [36] R. Ferede, G.C.H.E. de Croon, C. de Wagter y D. Izzo.
End-to-end neural network based optimal quadcopter control.
Robotics and Autonomous Systems, vol. 172, 2024. URL:
<https://resolver.tudelft.nl/uuid:16169a19-bf6b-4681-8ecc-18a9f2bd5e0f>.
- [37] H. Weber.
Autonomous Drone Navigation Using Reinforcement Learning Algorithms.
Scientific Journal of Artificial Intelligence and Blockchain Technologies, vol. 2, n.º 3, 2025. DOI:
<https://doi.org/10.63345/sjaibt.v2.i3.303>. URL:
https://www.researchgate.net/publication/396807493_Autonomous_Drone_Navigation_Using_Reinforcement_Learning_Algorithms.
- [38] A. V. R. Katkuri, H. Madan, N. Khatri, A. S. H. Abdul-Qawy y K. S. Patnaik.
Autonomous UAV navigation using deep learning-based computer vision frameworks: A systematic literature review.
Array, vol. 23, 2024, artículo 100361. DOI:
<https://doi.org/10.1016/j.array.2024.100361>. URL:
<https://www.sciencedirect.com/science/article/pii/S2590005624000274>.

BIBLIOGRAFÍA

- [39] M. Guazzaloca.
Learning-Based Control for Nano-Drones Flight: From Simulation to Reality.
Master Thesis, Alma Mater Studiorum – University of Bologna, Academic Year 2024/2025. URL: <https://amslaurea.unibo.it/id/eprint/36244/1/main.pdf>.
- [40] SharkYun.
Computer Vision — Object Detection, One-Stage vs Two-Stage detectors.
2024. URL: <https://sharkyun.medium.com/computer-vision-object-detection-one-stage-vs-two-stage-b05dbff88195>.
- [41] Abirami Vina.
¿Qué es R-CNN? Una descripción general rápida.
Ultralytics, 7 de junio de 2024. URL: <https://www.ultralytics.com/es/blog/what-is-r-cnn-a-quick-overview>.
- [42] Abirami Vina.
¿Qué es R-CNN? Una descripción general rápida.
Ultralytics, 7 de junio de 2024. URL: <https://www.ultralytics.com/es/blog/what-is-r-cnn-a-quick-overview>.
- [43] P. F. Felzenszwalb, R. B. Girshick, D. McAllester y D. Ramanan.
Object Detection with Discriminatively Trained Part-Based Models.
IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 32, n.º 9, 2010, pp. 1627–1645. URL: <https://cs.brown.edu/people/pfelzens/papers/lsvm-pami.pdf>.
- [44] J. Redmon, S. Divvala, R. Girshick y A. Farhadi.
You only look once: Unified, real-time object detection.
CVPR, 2016. URL: <http://arxiv.org/abs/1506.02640>.
- [45] J. Redmon y A. Farhadi.
YOLOv3: An Incremental Improvement.
arXiv preprint arXiv:1804.02767, 2018. URL: <https://arxiv.org/abs/1804.02767>.
- [46] DataCamp.
Explicación de la detección de objetos YOLO: Guía para principiantes.
29 de enero de 2024. URL: <https://www.datacamp.com/es/blog/yolo-object-detection-explained>.

BIBLIOGRAFÍA

- [47] Ultralytics.
Predicción de Modelos con Ultralytics YOLO.
Documentación de Ultralytics YOLO, consultado el 19 de abril de 2026. URL:
<https://docs.ultralytics.com/es/modes/predict/>.
- [48] J. Terven et al.
A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS.
2023. URL: <https://www.mdpi.com/2504-4990/5/4/83>.
- [49] AIknow.io.
Introducción a YOLO.
2024. URL:
<https://www.aiknow.io/es/introduccion-a-yolo/>.
- [50] Y. Zheng et al.
Air-to-air visual detection of micro-UAVs: An experimental evaluation of deep learning.
IEEE Robotics and Automation Letters, 2021. URL:
<https://ieeexplore.ieee.org/document/9343737>.
- [51] A. Lukezic et al.
Discriminative correlation filter with channel and spatial reliability.
CVPR, 2017. URL:
<http://ieeexplore.ieee.org/document/8099998/>.
- [52] R. E. Kalman.
A new approach to linear filtering and prediction problems.
1960. URL: <https://asmedigitalcollection.asme.org/fluidsengineering/article/82/1/35/397706/A-New-Approach-to-Linear-Filtering-and-Prediction>.
- [53] KalmanFilter.net.
Filtro de Kalman explicado mediante ejemplos.
Consultado el 20 de abril de 2026. URL:
https://kalmanfilter.net/ES/default_es.aspx.
- [54] Wikipedia.
Filtro de Kalman.
Consultado el 20 de abril de 2026. URL:
https://es.wikipedia.org/wiki/Filtro_de_Kalman.

BIBLIOGRAFÍA

- [55] Industrial Monitor Direct.
Analysis of an Exponential Moving Average (EMA) Filter in Ladder Logic for Load Cell Signal Conditioning.
Consultado el 20 de abril de 2026. URL: <https://industrialmonitordirect.com/blogs/knowledgebase/analysis-of-an-exponential-moving-average-ema-filter-in-ladder-logic-for-load->
- [56] Wikipedia.
Suavizamiento exponencial.
Consultado el 20 de abril de 2026. URL: https://es.wikipedia.org/wiki/Suavizamiento_exponencial.
- [57] MCI Electronics.
Filtro exponencial EMA (Exponential Moving Average).
Consultado el 20 de abril de 2026. URL: <https://cursos.mcielectronics.cl/2025/07/24/filtro-exponencial-ema-exponential-moving-average/>.
- [58] H. Jabbari Asl, G. Oriolo y H. Bolandi.
An adaptive scheme for image-based visual servoing of an underactuated UAV.
Int. Journal of Robotics and Automation, vol. 29, 2014. DOI: https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://www.researchgate.net/publication/260173544_An_adaptive_scheme_for_image-based_visual_servoing_of_an_underactuated_uav&ved=2ahUKEwiCisPGve2TAxVnxgIHHTv8DRkQFnoECBkQAQ&usg=AOvVaw23lbIYBAzu8TNIdwwOtQ4n.
- [59] OpenCV.
Camera Calibration and 3D Reconstruction.
URL: https://docs.opencv.org/4.x/d9/d0c/group__calib3d.html.
- [60] Wikipedia.
Cámara estenopeica.
Consultado el 4 de mayo de 2026. URL: https://es.wikipedia.org/wiki/C%C3%A1mara_estenopeica.
- [61] Wikimedia Commons.
File:Pinhole-camera.svg.
Imagen esquemática del funcionamiento de una cámara estenopeica, utilizada en el artículo de Wikipedia *Cámara estenopeica*. URL: <https://commons.wikimedia.org/wiki/File:Pinhole-camera.svg>.

- [62] openMVG library.
cameras.
Documentación del modelo de cámara *pinhole*, de donde se obtiene el esquema utilizado en esta memoria. Consultado el 4 de mayo de 2026. URL: <https://openmvg.readthedocs.io/en/latest/openMVG/cameras/cameras/>.
- [63] CameraModels.
Camera Models.
URL: https://web.stanford.edu/class/cs231a/course_notes/01-camera-models.pdf.
- [64] F. Bergamasco.
Pinhole Camera.
Computer Vision, curso 2018/2019. URL: https://www.dsi.unive.it/~bergamasco/teachingfiles/cvslides2019/13_pinhole_camera.pdf.
- [65] R. van den Boomgaard.
The Pinhole Camera.
Lecture Notes on Image Processing and Computer Vision, University of Amsterdam. URL: <https://staff.fnwi.uva.nl/r.vandenboomgaard/IPCV20162017/LectureNotes/CV/PinholeCamera/PinholeCamera.html>.
- [66] Disorsiones de una camara pinhole.
Disorsiones de una camara pinhole.
URL: https://www.researchgate.net/figure/A-pinhole-camera-shows-no-distortion-Images-of-a-rectangular-object-screen-shows-distortion-fig2_316300680.
- [67] Esquema del PID.
Esquema del PI.
URL: <https://controlautomaticoeducacion.com/control-de-procesos/control-ipd/>.
- [68] Dewesoft.
¿Qué es un controlador PID?.
URL: <https://dewesoft.com/es/blog/que-es-un-controlador-pid>.
- [69] Wikipedia.
Controlador PID.
URL: https://es.wikipedia.org/wiki/Controlador_PID.